

DS5899: Special Topics in Data Science

Leveraging NLP in Asset Management

Week 2: Attention

Instructor: Che Guan, Ph.D.

Part 1:

Transformer

Transformer Architecture

BERT

Encoder

GPT

Decoder

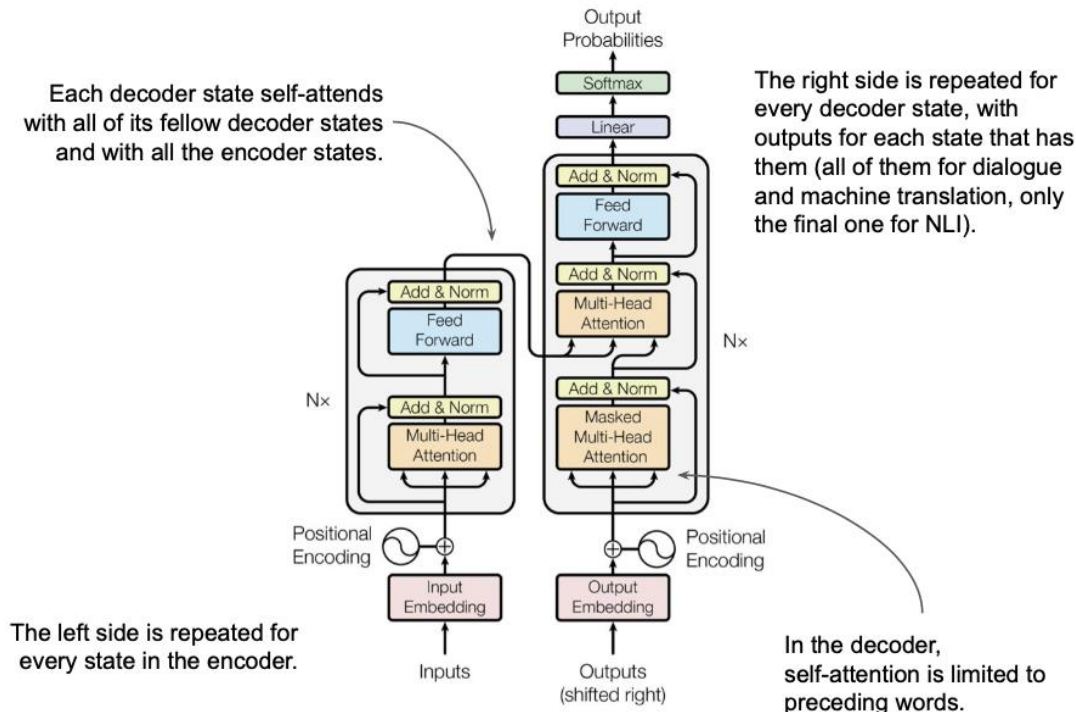
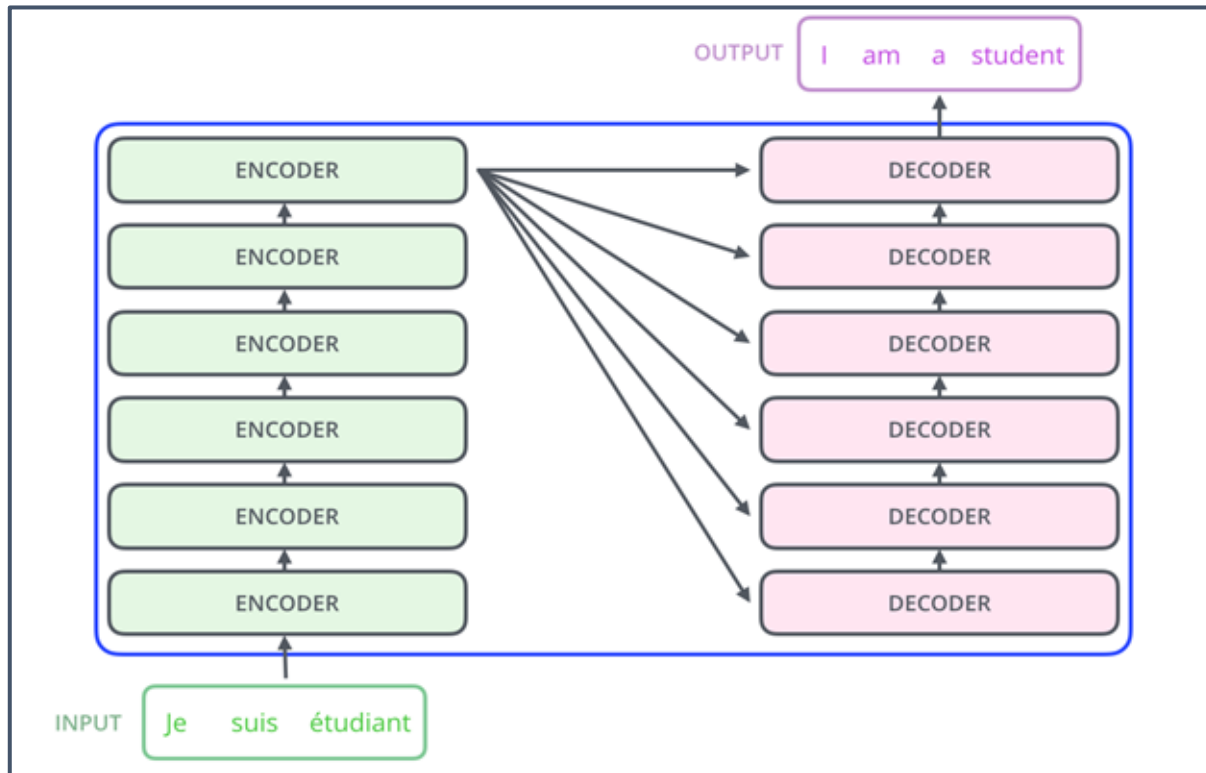


Figure 1: The Transformer - model architecture.

Transformer, *Continued*

- Transformer is composed of 2 main stacks: an **Encoder** (which processes input) and a **Decoder** (which generates output):
 - **Encoder-Decoder (Original, T5)**: The Encoder processes the source (e.g., English), and the Decoder attends to the Encoder's output to generate the target (e.g., French).
 - **Encoder-Only (BERT)**: Uses only the first half. It can look at the whole sentence at once (bidirectional). Good for classification and search.
 - **Decoder-Only (GPT, Llama, Claude)**: Uses only the second half with "Masked" attention (it can't see the future words). Good for generating text.

Transformer, *Continued*



Transformer, *Continued*

1. The Input Stage

- **Embeddings:** The raw text is tokenized and converted into vectors.
- **Positional Encoding:** Transformers process all words in parallel.

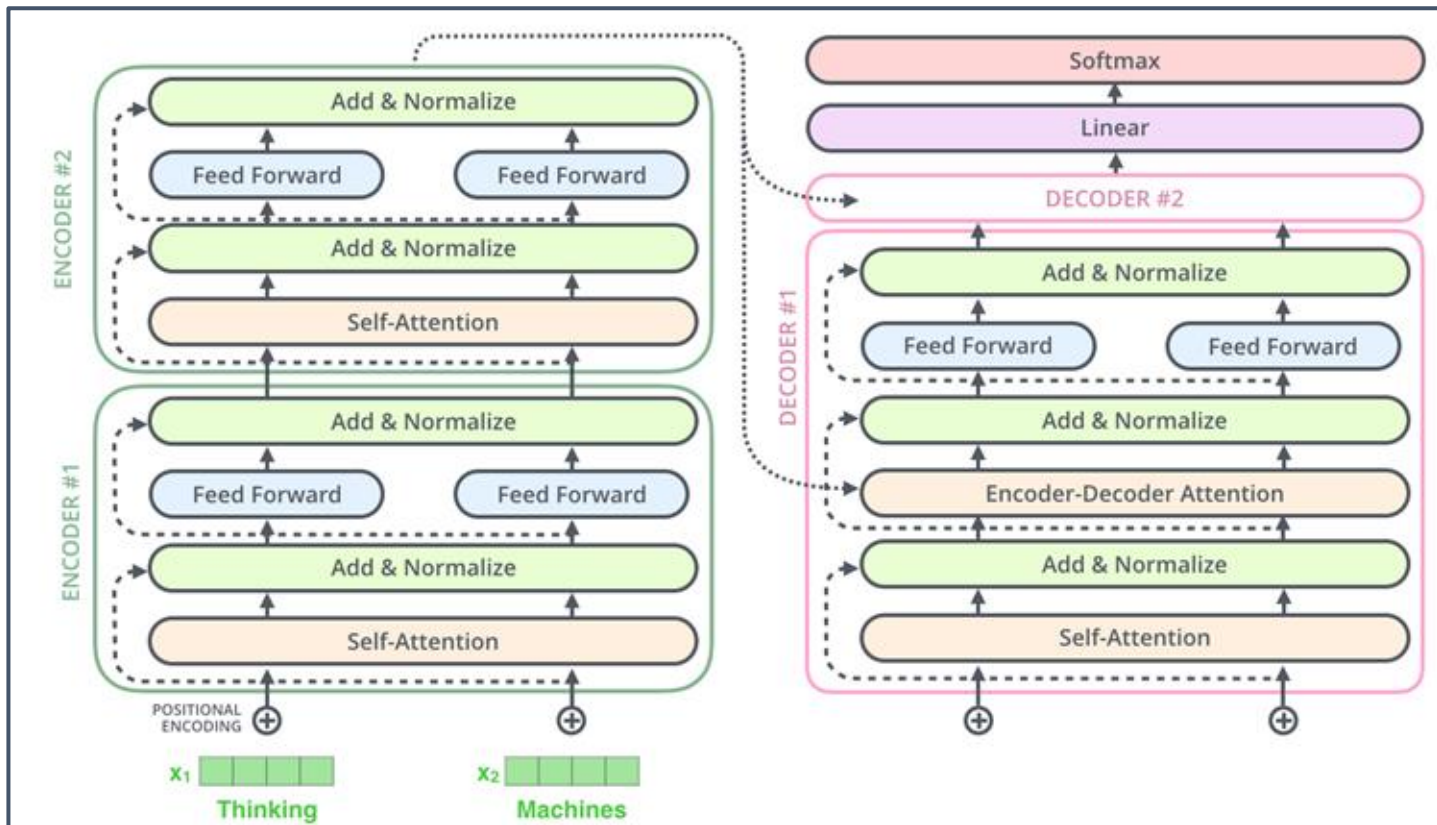
2. The Transformer Block (The Repeating Unit)

- **Multi-Head Self-Attention:** aggregates context from input tokens
- **Add & Norm (Layer Normalization):** stabilizes the signal
- **Feed-Forward Network (FFN):** processes features individually

3. The Output Stage (for Decoders):

- The final vector is projected to a probability distribution to predict the next token.

Transformer Zoom In



Encoder

- The residual connection (the "Add" in "Add & Norm") provides a direct "superhighway" for information and gradients to flow through the network and hence prevents Vanishing Gradients
- Layer Normalization (LayerNorm) forces the input vector to have a Mean of 0 and a Variance of 1

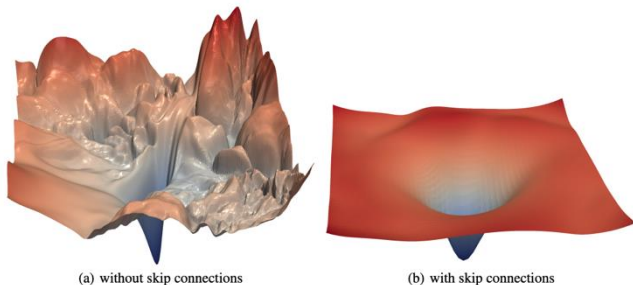
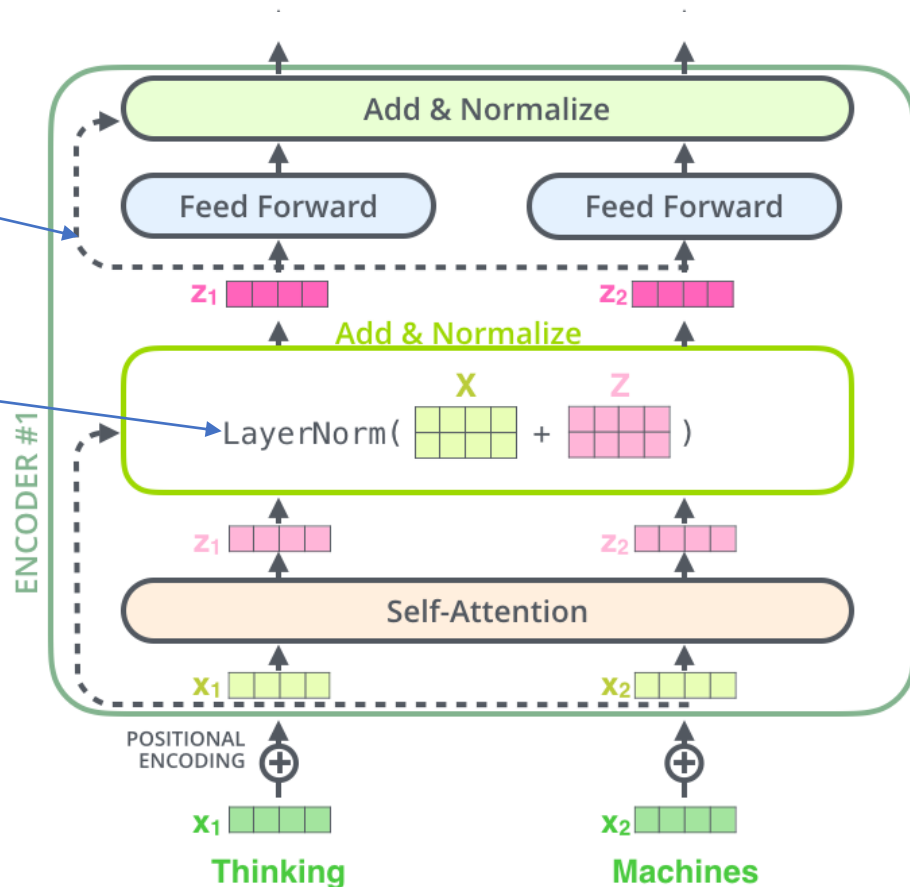
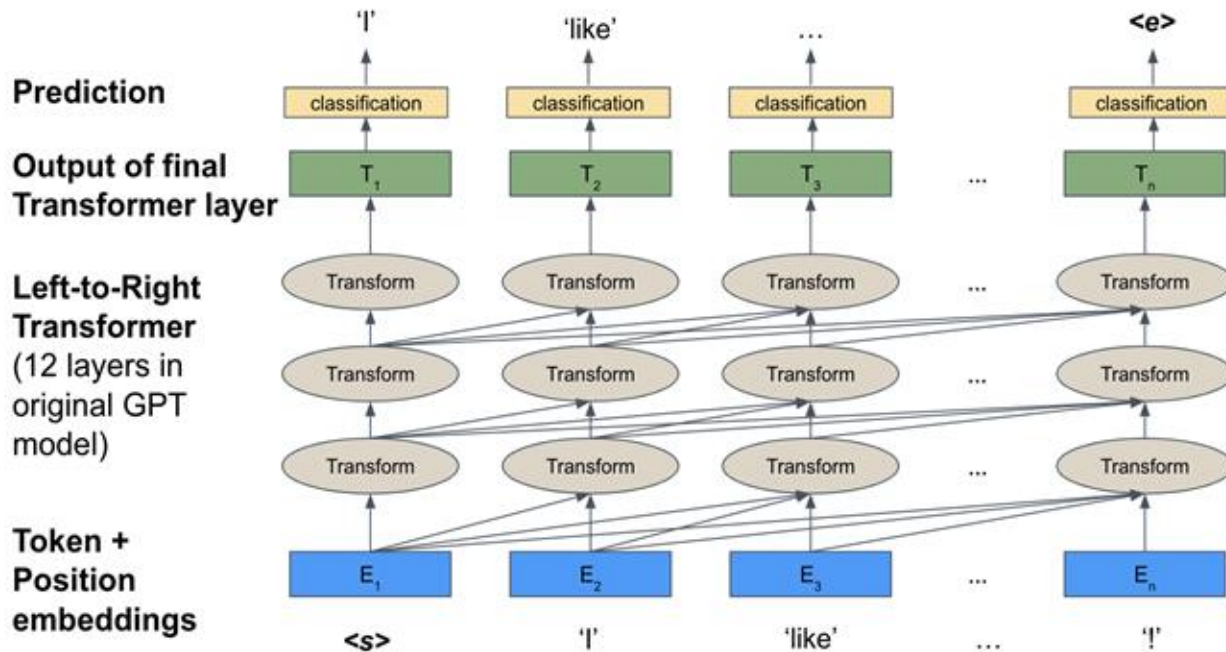


Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.



Decoder

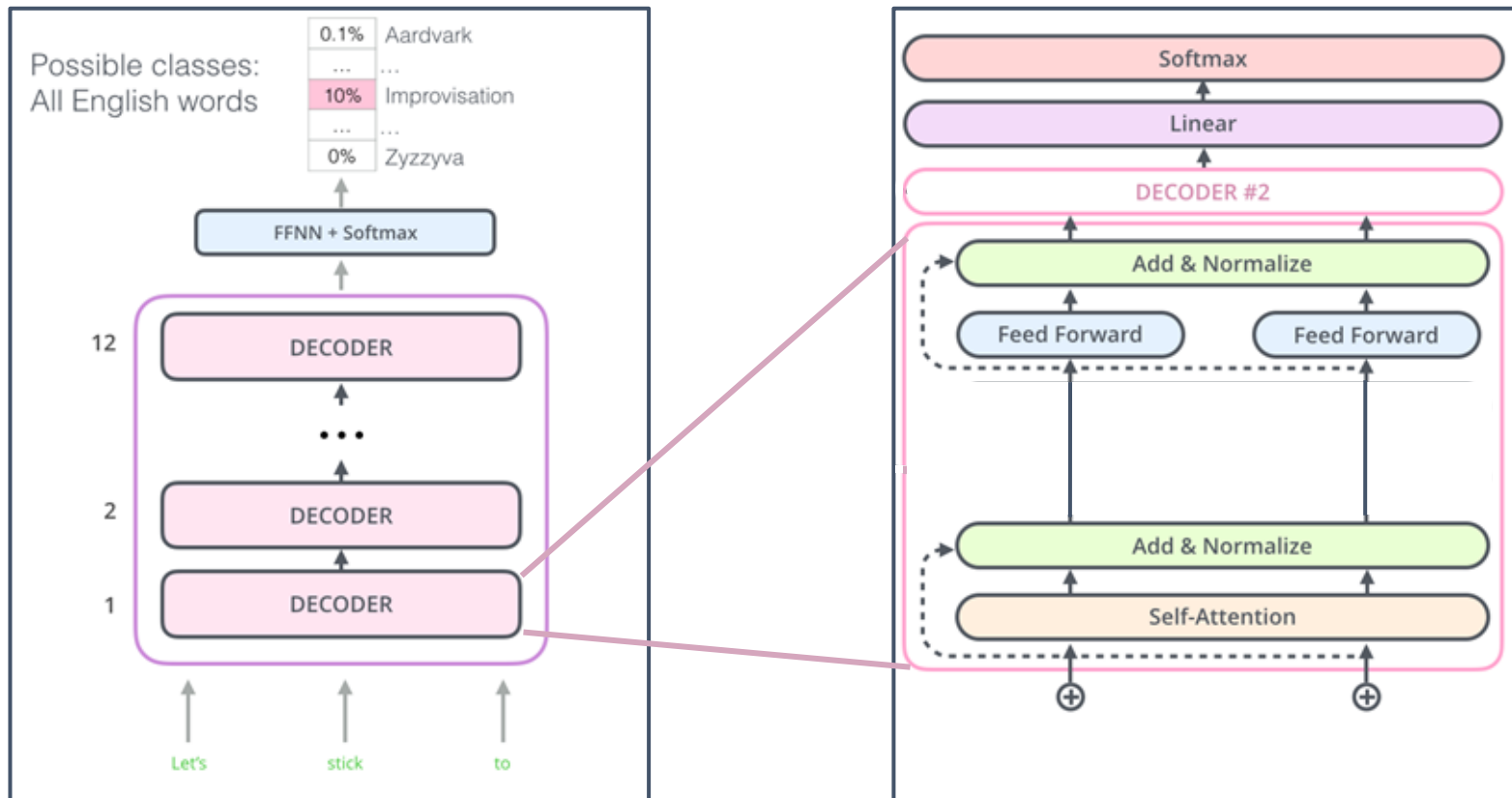
Language Model Task



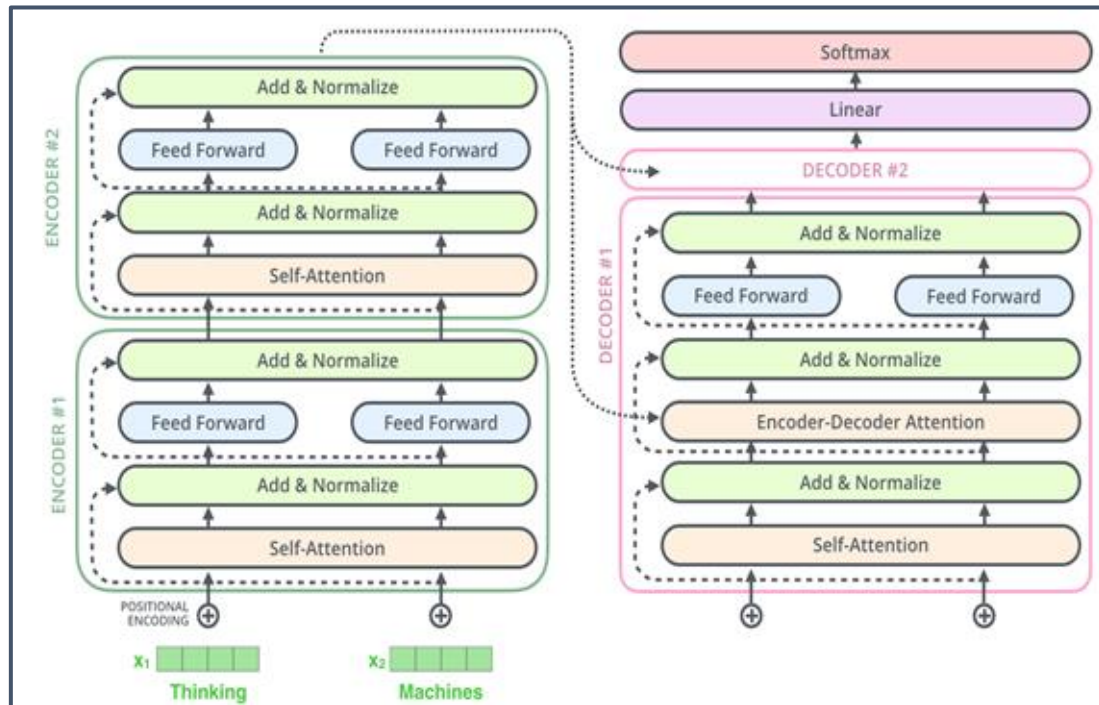
To enable parallelization, we **mask out attention** to future words by setting attention scores to $-\infty$.

$$e_{ij} = \begin{cases} q_i^T k_j, & j < i \\ -\infty, & j \geq i \end{cases}$$

Decoder *Continued*



Encoder-Decoder



- The “Encoder-Decoder Attention” layer works just like multiheaded self-attention, except it creates its **Queries matrix from the layer below it**, and takes the Keys and Values matrix from the output of the encoder stack.

[Image source: Jay Alammar, The Illustrated Transformer](#)

Understand Query, Key and Value

- Consider the sentence: "**The bank of the river.**" where "bank" could mean a financial institution or the side of a river.
- **The Query (from "bank"):**
 - "bank" creates a **Query** that effectively asks: *"I need context! Are there any words here related to money or nature?"*
- **The Key (from other words):**
 - The word "The" holds up a **Key**: *"I am a common article."* (Low relevance).
 - The word "of" holds up a **Key**: *"I am a preposition."* (Low relevance).
 - The word "river" holds up a **Key**: *"I am a nature object, specifically water."* (High!)
- **The Value (from 'river'):**
 - Since a match was found, the word "river" delivers its **Value**: *"Here is the concept of water and nature."*

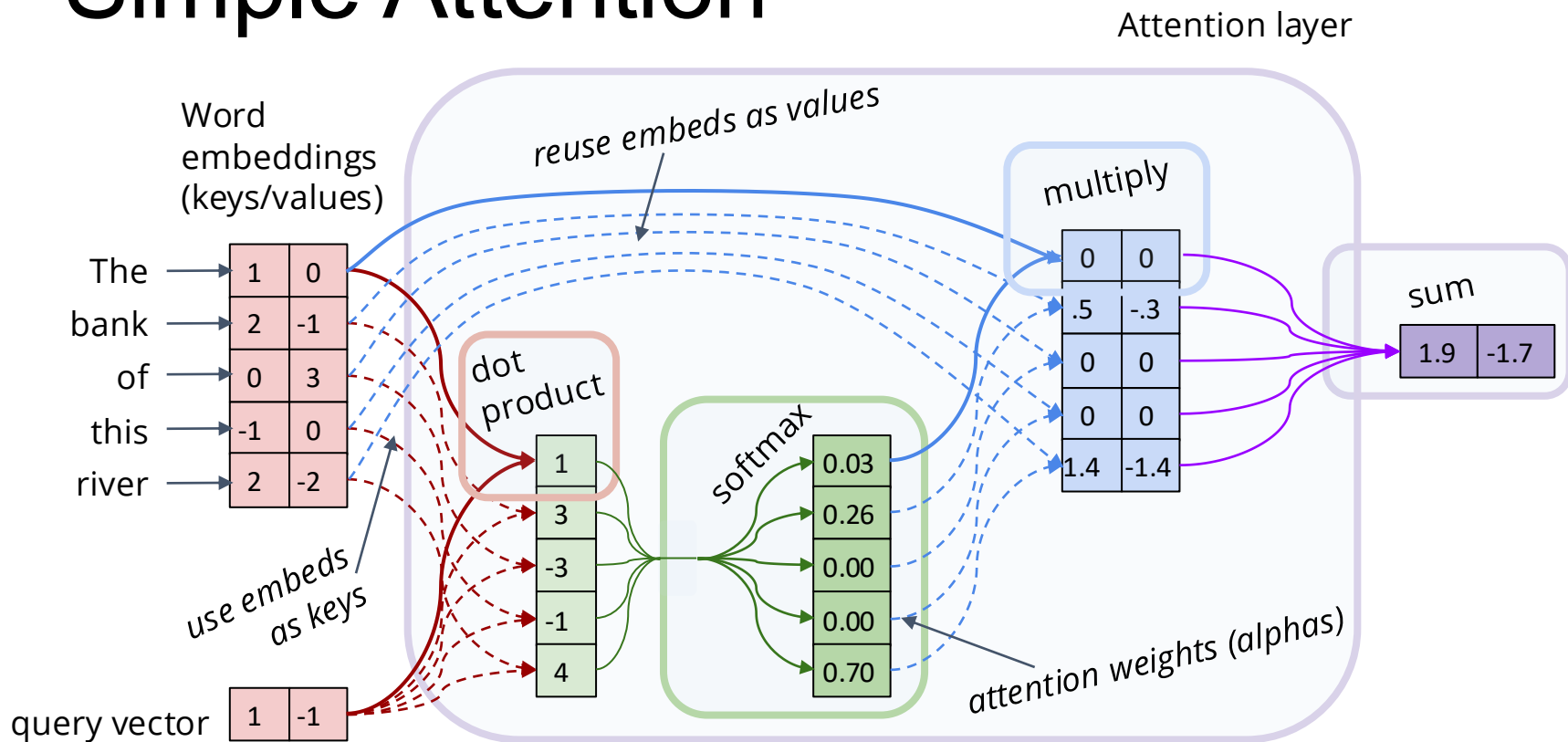
Query, Key and Value, *Continued*

- In a standard Transformer Self-Attention layer, both Key and Value originate from the same input X, If they have the same size, what is the reasoning behind splitting them?
- The answer lies in **Separation of Concerns**.
 - The model needs to separate "**Addressing**" (how I find you) from "**Content**" (what information you give me).

A Simple Analogy

- Imagine you are writing a query for a movie recommendation:
 - the Key matrix learns to extract "Genre features" (Action, Dark)
 - the Value matrix learns to extract "Content features" (Bruce Wayne, Gotham, 9/10 stars)
 - This allows the model: *"Match based on genre, but return the plot."*
- Strictly speaking, K and V do not have to have the same dimension. Q and K must have the same dimension.

Simple Attention



Self-Attention (1-Layer) Example

<i>Embedding E</i> <i>(from Token + Position)</i>	<table><tr><td>1</td></tr><tr><td>12</td></tr><tr><td>3</td></tr><tr><td>-1</td></tr></table>	1	12	3	-1	<table><tr><td>-2</td></tr><tr><td>-5</td></tr><tr><td>3</td></tr><tr><td>3</td></tr></table>	-2	-5	3	3	<table><tr><td>2</td></tr><tr><td>2</td></tr><tr><td>-2</td></tr><tr><td>7</td></tr></table>	2	2	-2	7	<table><tr><td>2</td></tr><tr><td>1</td></tr><tr><td>-2</td></tr><tr><td>-6</td></tr></table>	2	1	-2	-6	<table><tr><td>7</td></tr><tr><td>-1</td></tr><tr><td>12</td></tr><tr><td>3</td></tr></table>	7	-1	12	3
1																									
12																									
3																									
-1																									
-2																									
-5																									
3																									
3																									
2																									
2																									
-2																									
7																									
2																									
1																									
-2																									
-6																									
7																									
-1																									
12																									
3																									
<i>Sentence</i>	'The'	'bank'	'of'	'this'	'river'																				

Self-Attention (1-Layer)

Weight Matrices

W^Q, W^K, W^V

Learned weight matrices

W^Q		W^K		W^V	
0.34	-0.11	-0.13	-0.50	0.05	0.32
-0.12	0.12	-0.11	0.11	0.22	0.09
-0.29	-0.39	0.29	0.45	-0.26	-0.14
0.01	-0.09	-0.07	-0.39	-0.02	0.61

Embedding E

(from Token + Position)

Sentence

1	-2	2	2	7
12	-5	2	1	-1
3	3	-2	-2	12
-1	3	7	-6	3
'The'	'bank'	'of'	'this'	'river'

Self-Attention (1-Layer)

**Queries, Keys,
Values**

$(E \times W^Q, E \times W^K, E \times W^V)$

Q ₁	K ₁	V ₁	Q ₂	K ₂	V ₂	Q ₃	K ₃	V ₃	Q ₄	K ₄	V ₄	Q ₅	K ₅	V ₅
1	-1	3	2	1	-1	4	-2	2	4	-1	2	2	2	-2
1	3	-1	-1	1	-1	1	-4	4	2	1	-4	-5	1	1

Weight Matrices

W^Q, W^K, W^V

*Learned weight matrices,
multiply each matrix
by each embedding
to get each token's
Q, K and V vectors*

W^Q	W^K	W^V
0.34	-0.13	0.05
-0.12	-0.11	0.32
-0.29	0.29	0.22
0.01	-0.07	-0.26
-0.11	-0.39	0.09
0.12	0.45	-0.14
-0.39	-0.39	0.61

Embedding E

(from Token + Position)

1
12
3
-1

-2
-5
3
3

2
2
-2
7

2
1
-2
-6

7
-1
12
3

Sentence

'The'

'bank'

'of'

'this'

'river'

Self-Attention (1-Layer)

Attention Weights

($\alpha = \text{softmax}(s)$)

Dot Product Scores s

($s = Q_i \cdot K_j$ for all i, j)

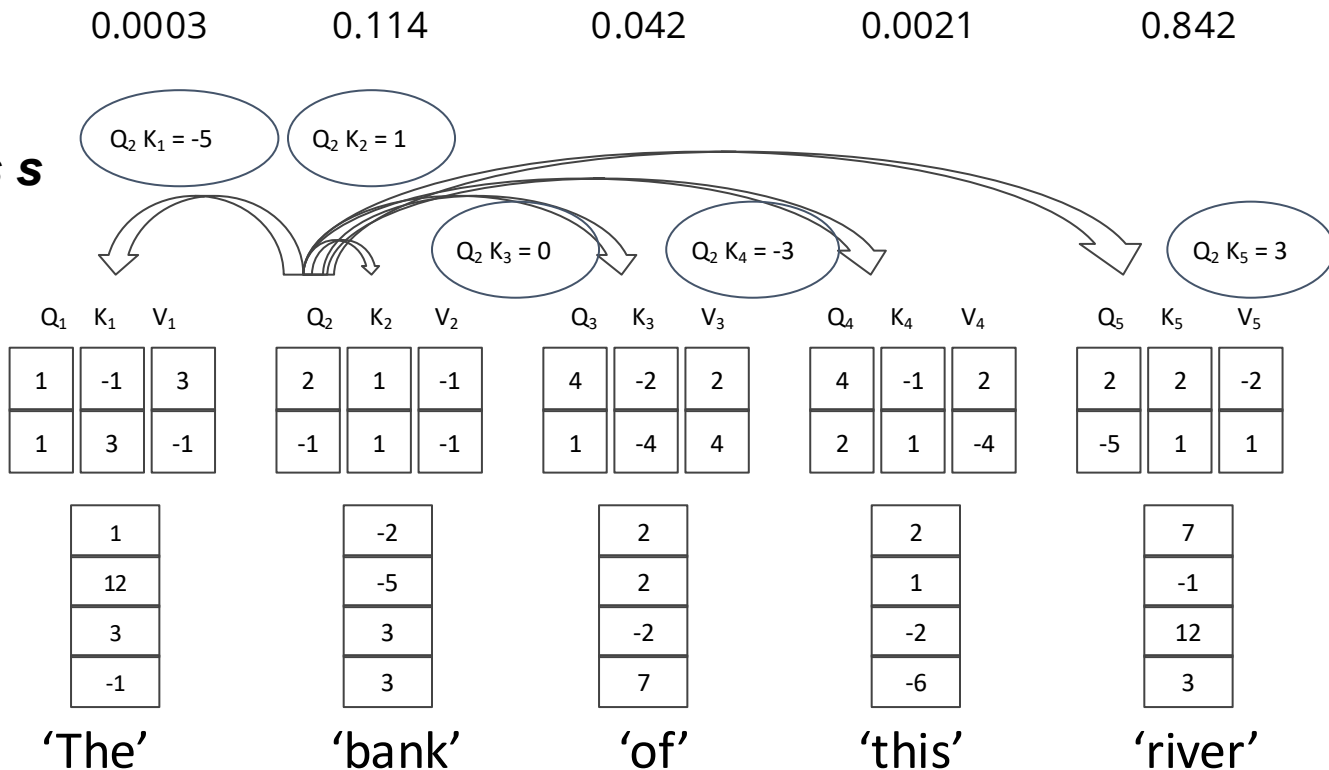
Queries, Keys, Values

($E \times W^Q, E \times W^K, E \times W^V$)

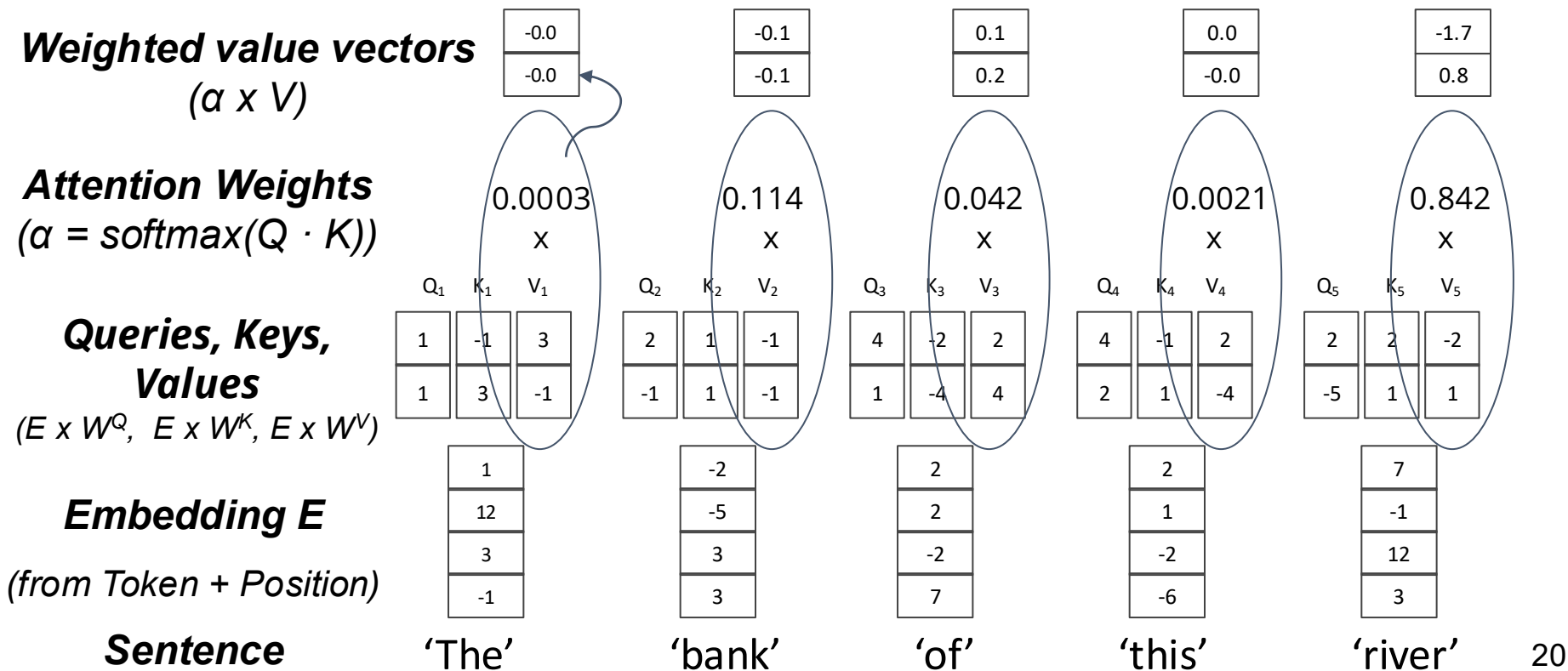
Embedding E

(from Token + Position)

Sentence



Self-Attention (1-Layer)



Self-Attention (1-Layer)

Context vector

$$\text{sum}(\alpha \times V)$$

-1.7
0.9

The attention vector for the word 'bank'

+

Attention Weights

$$(\alpha = \text{softmax}(Q \cdot K))$$

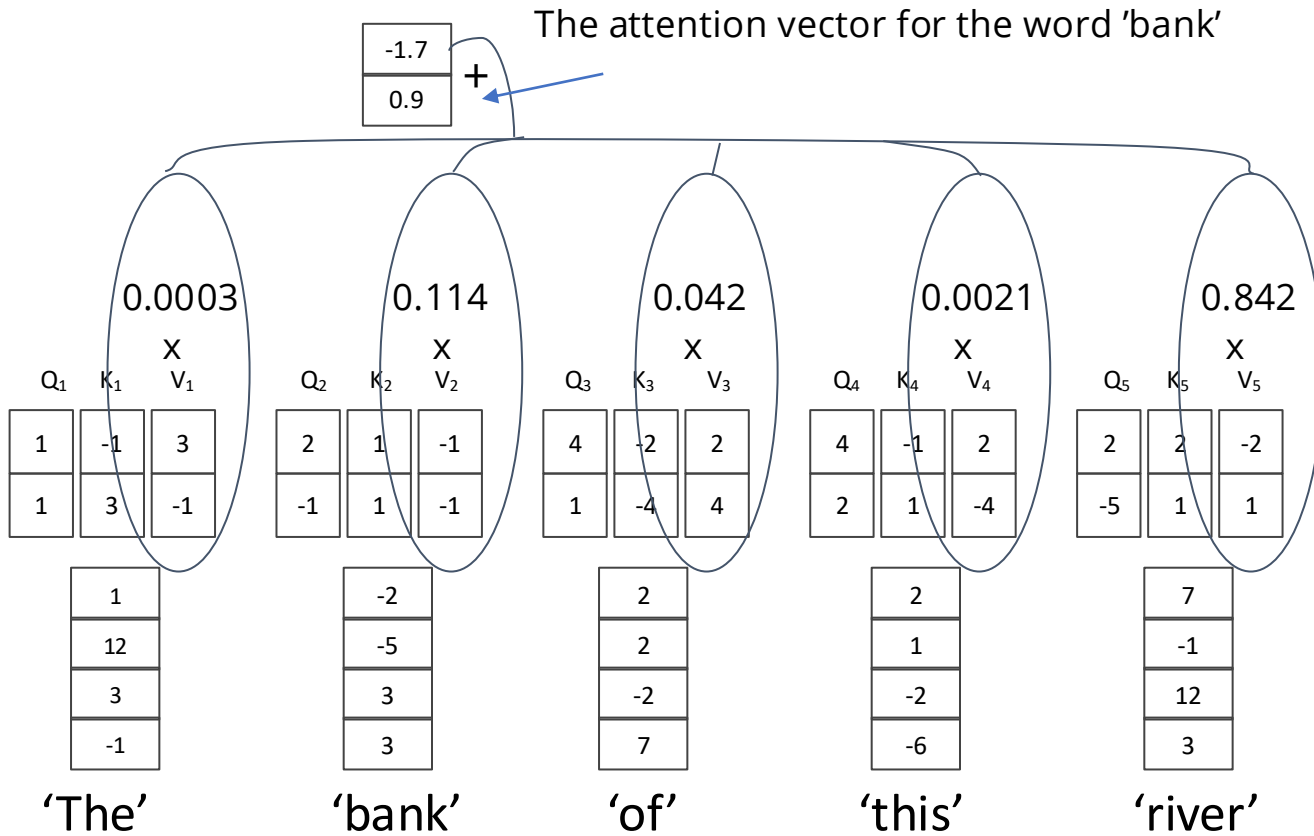
**Queries, Keys,
Values**

$$(E \times W^Q, E \times W^K, E \times W^V)$$

Embedding E

(from Token + Position)

Sentence



Attention

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

=

Z

$\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}$

Multi-Head Attention

- In **Multi-Head Attention**, the projected vectors are not kept as one giant vector. They are split into smaller "heads" to allow the model to focus on different aspects of the input simultaneously:
 - **The Operation:** The vector of size d_{model} (e.g., 512) is reshaped into h heads of size d_{head} (e.g., 8 heads x 64 dimension).
 - The Transformation: $[Batch, SeqLen, d_{model}] \rightarrow [Batch, SeqLen, Heads, d_{head}]$
- Example:
 - Sentence A: "I love AI." (3 tokens)
 - Sentence B: "Transformers are very powerful." (4 tokens)
 - The Input Tensor:

[I]	[love]	[AI]	[PAD]
[Transformers]	[are]	[very]	[powerful]

Batch (Size): 2 (Rows)
SeqLen: 4 (Columns)

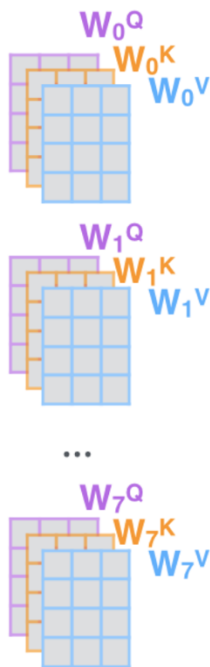
Multi-Head Attention

1) This is our
input sentence*

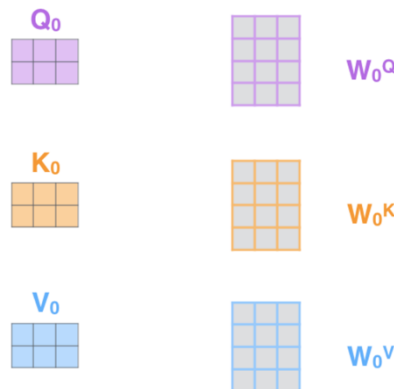
2) We embed
each word*

3) Split into 8 heads.
We multiply X or
 R with weight matrices

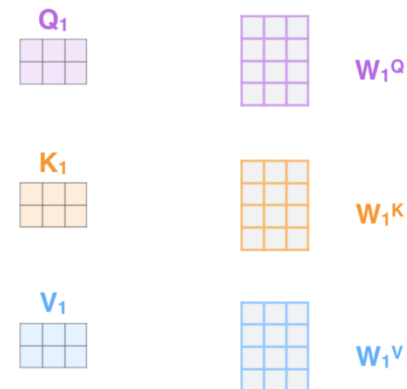
Thinking
Machines



ATTENTION HEAD #0



ATTENTION HEAD #1



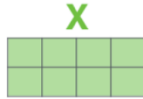
[Image Source: Jay Alammar, The Illustrated Transformer](#)

Multi-Head Attention

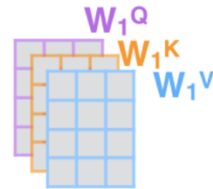
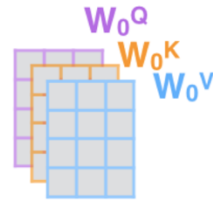
1) This is our input sentence*

Thinking
Machines

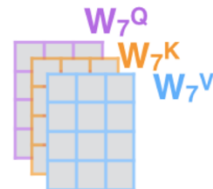
2) We embed each word*



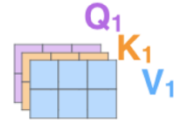
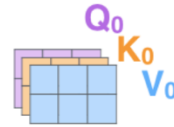
3) Split into 8 heads.
We multiply X or R with weight matrices



...



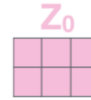
4) Calculate attention using the resulting $Q/K/V$ matrices



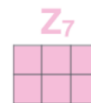
...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



...



[Image Source: Jay Alammar, The Illustrated Transformer](#)

Multi-Head Attention

1) This is our input sentence*

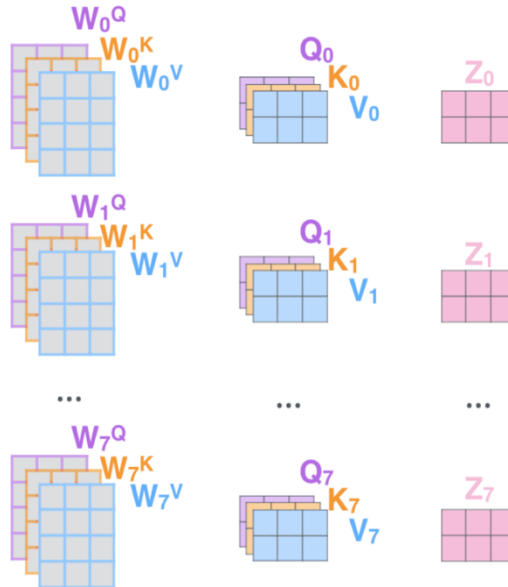
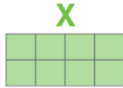
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking Machines



1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$

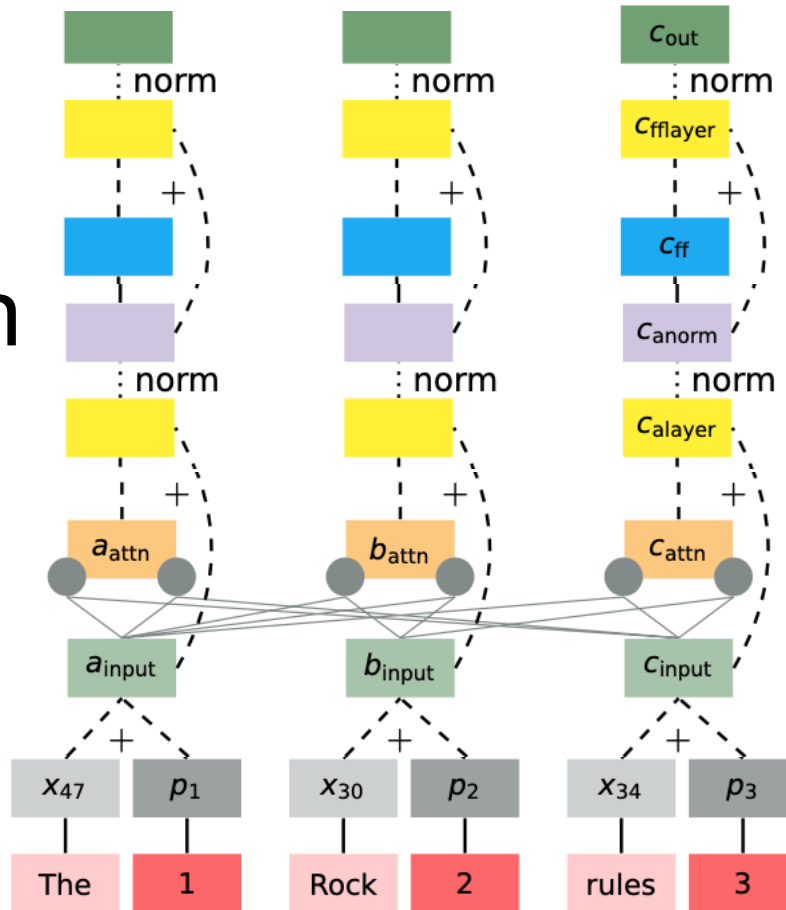


3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



[Image Source: Jay Alammar. The Illustrated Transformer](#)

Revisit Attention Calculation



$$C_{out} = \frac{C_{fflayer} - \text{mean}(C_{fflayer})}{\text{std}(C_{fflayer}) + \epsilon}$$

$$C_{fflayer} = C_{anorm} + \text{Dropout}(C_{ff})$$

$$C_{ff} = \text{ReLU}(C_{anorm}W_1 + b_1)W_2 + b_2$$

$$C_{anorm} = \frac{C_{alayer} - \text{mean}(C_{alayer})}{\text{std}(C_{alayer}) + \epsilon}$$

$$C_{alayer} = \text{Dropout}(C_{attn} + C_{input})$$

$$C_{attn} = \text{sum}([\alpha_1 a_{input}, \alpha_2 b_{input}])$$

$$\alpha = \text{softmax}(\tilde{\alpha})$$

$$\tilde{\alpha} = \left[\frac{C_{input}^T a_{input}}{\sqrt{d_k}}, \frac{C_{input}^T b_{input}}{\sqrt{d_k}} \right]$$

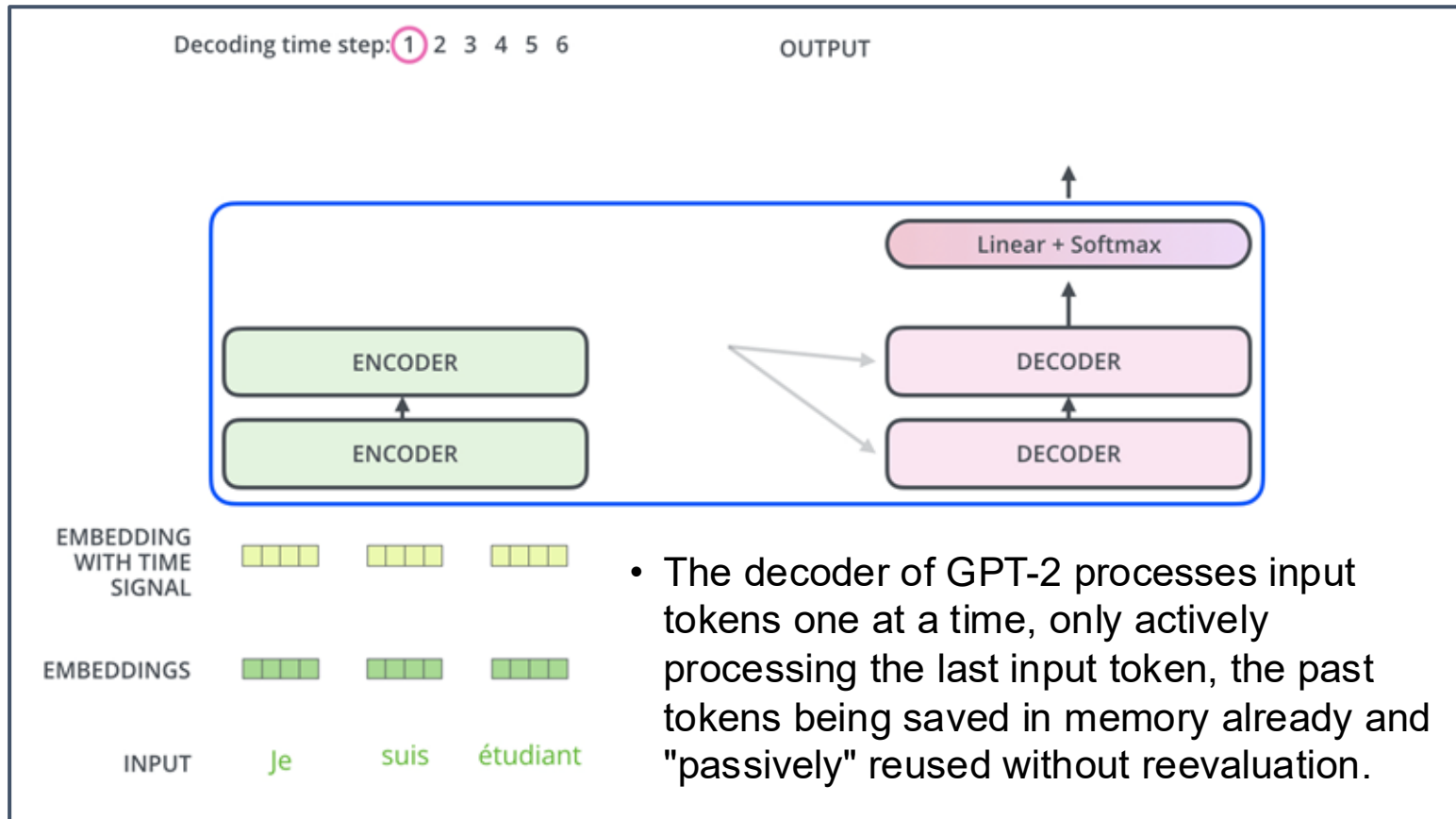
$$C_{input} = x_{34} + p_3$$

Note that C_{input} is not attended to itself in this example.

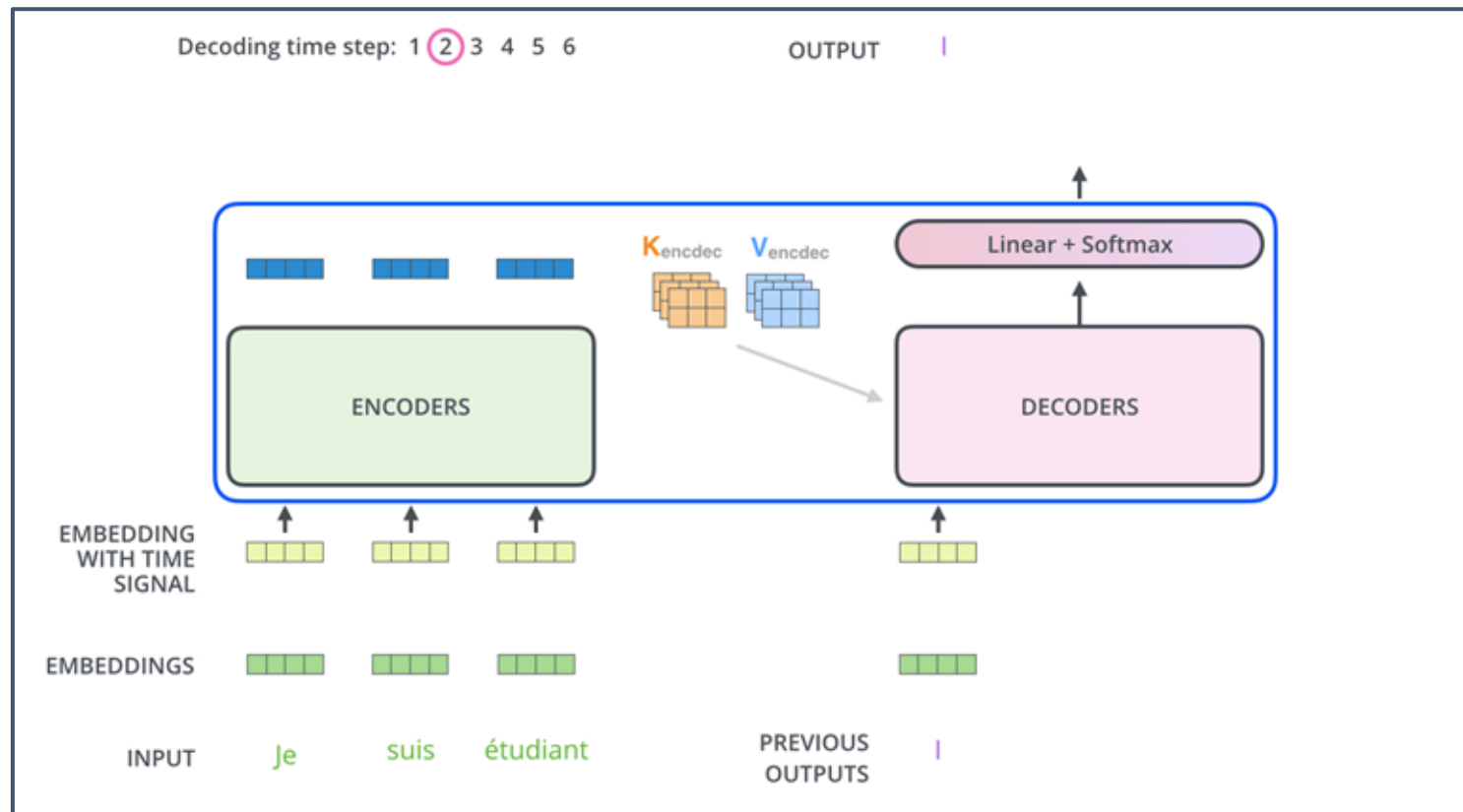
Part 2:

Attention

Inferencing



Inferencing, *Continued*



An Illustration of Key-Value Caching

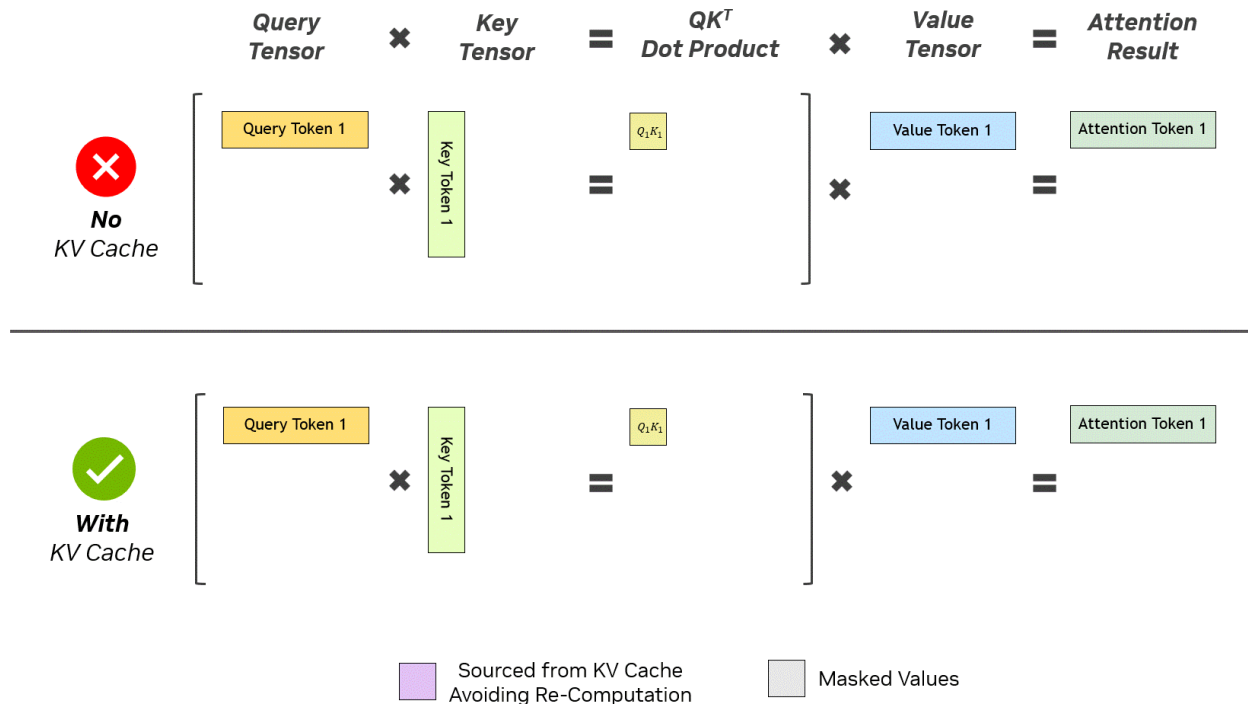
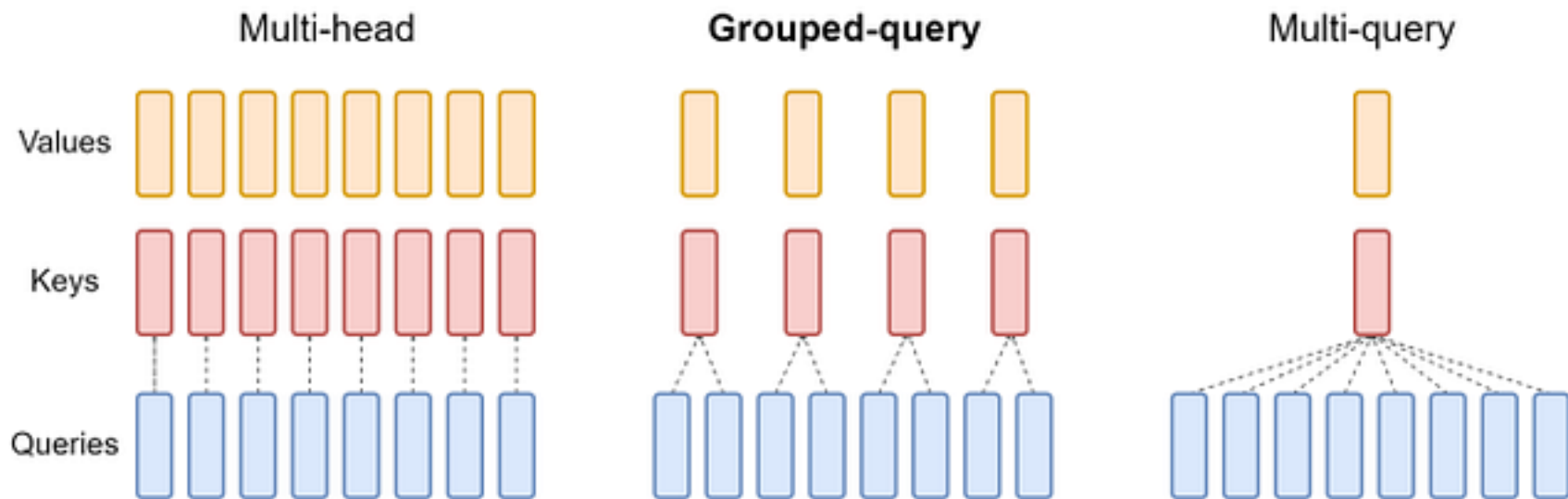
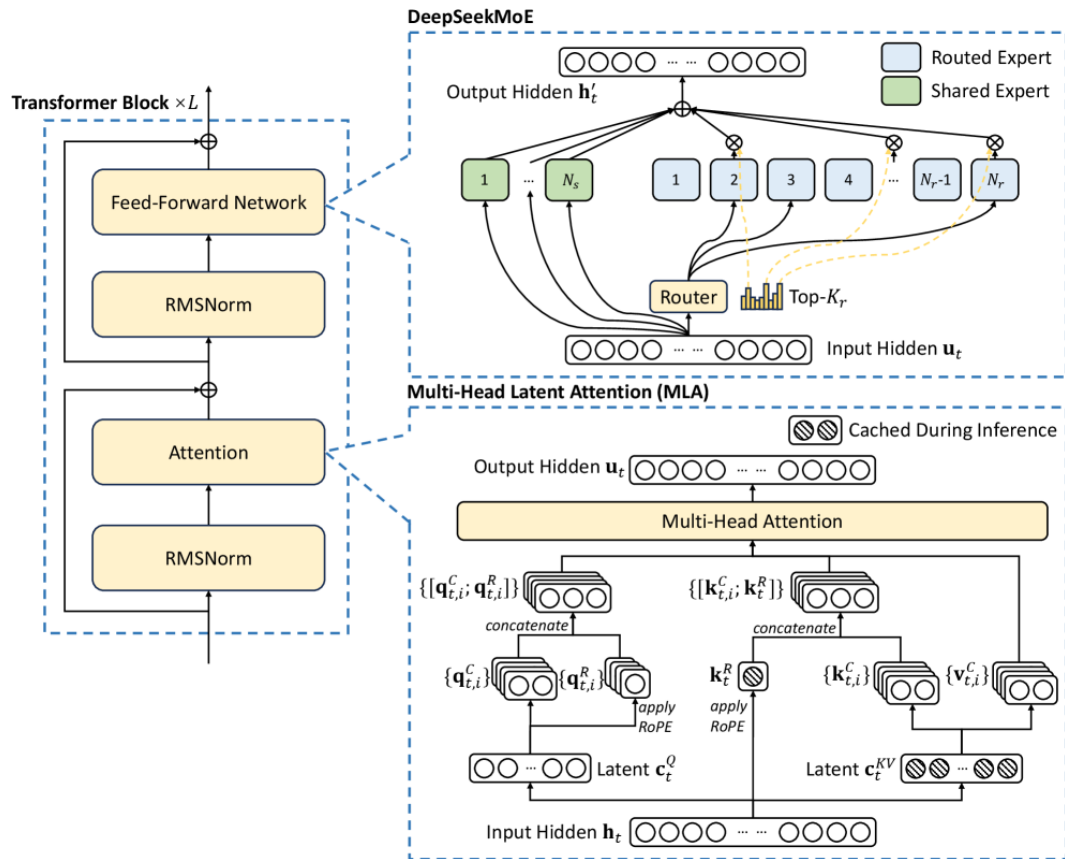


Figure 1. The top panel, labeled “No KV Cache,” shows that every new step recomputes queries, keys, values, and the full attention output for all tokens seen so far. The bottom panel, labeled “With KV Cache,” shows that only the current token’s query is newly computed, while all past keys and values are loaded from the cache, so the attention and output matrices are much smaller and redundant computation is avoided.

Multi-Query Attention (MQA) vs Grouped-Query Attention (GQA)



Multi-Head Latent Attention (MLA)



- The key idea of the **DeepSeekMoE** (Mixture of Experts) architecture (introduced in DeepSeek-V2 and V3) is to solve the inefficiencies of traditional MoE models by splitting experts into two distinct types: **Shared Experts** (for common knowledge) and **Fine-Grained Routed Experts** (for niche knowledge).
- Its key idea of MLA is to achieve the performance of standard Multi-Head Attention (MHA) while maintaining the low memory footprint (KV Cache) of Multi-Query Attention (MQA).

Image Resource: [Deepseek-v2](#)

ABSOLUTE POSITIONAL ENCODING (APE)

Position	1	2	3	4	5	6
	I	walk	my	dog	every	day

Position	1	2	3	4	5	6
	every	day	I	walk	my	dog

Limitation of APE

- The limitation for APE is that the model does not inherently know that "**I walk my dog**" is the same distinct object in both sentences.
- It treats the shifted version 'every day I walk my dog' as a **new pattern** that must be learned independently, simply because it "lives" at different addresses (indices) in the sequence.
- Rotary Positional Embeddings (RoPE) were designed specifically to solve the limitations of APE

Rotary Positional Embeddings (RoPE)

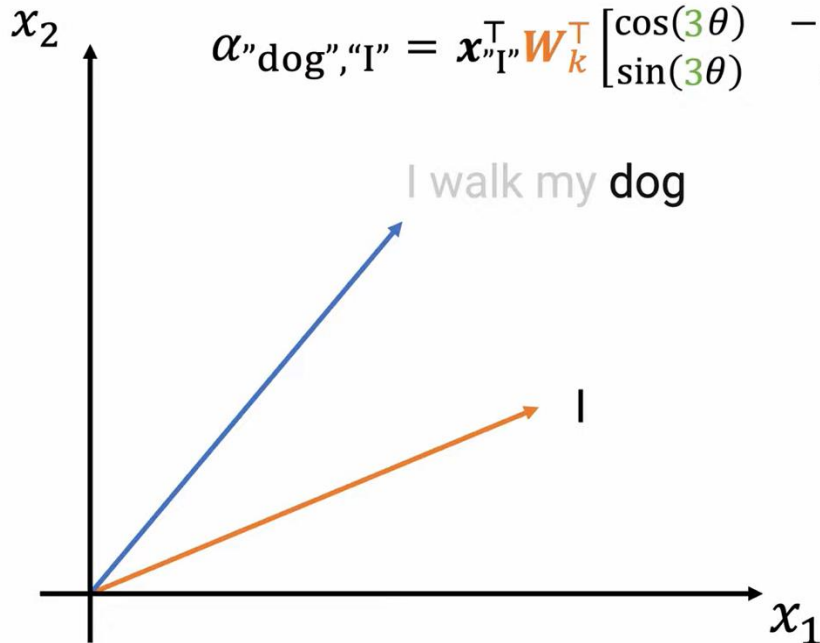
$$q_{\text{"dog"}} = \begin{bmatrix} \cos(4\theta) & -\sin(4\theta) \\ \sin(4\theta) & \cos(4\theta) \end{bmatrix} W_q x_{\text{"dog"}}$$

Query vector for "dog"

$$k_{\text{"I"}} = \begin{bmatrix} \cos(1\theta) & -\sin(1\theta) \\ \sin(1\theta) & \cos(1\theta) \end{bmatrix} W_k x_{\text{"I"}}$$

Key vector for "I"

$$\alpha_{\text{"dog","I"}} = x_{\text{"I"}}^T W_k^T \begin{bmatrix} \cos(3\theta) & -\sin(3\theta) \\ \sin(3\theta) & \cos(3\theta) \end{bmatrix} W_q x_{\text{"dog"}}$$



RoPE, Continued

$$q_{\text{"dog"}} = \begin{bmatrix} \cos(6\theta) & -\sin(6\theta) \\ \sin(6\theta) & \cos(6\theta) \end{bmatrix} W_q x_{\text{"dog"}}$$

Query vector for "dog"

$$k_{\text{"I"}} = \begin{bmatrix} \cos(3\theta) & -\sin(3\theta) \\ \sin(3\theta) & \cos(3\theta) \end{bmatrix} W_k x_{\text{"I"}}$$

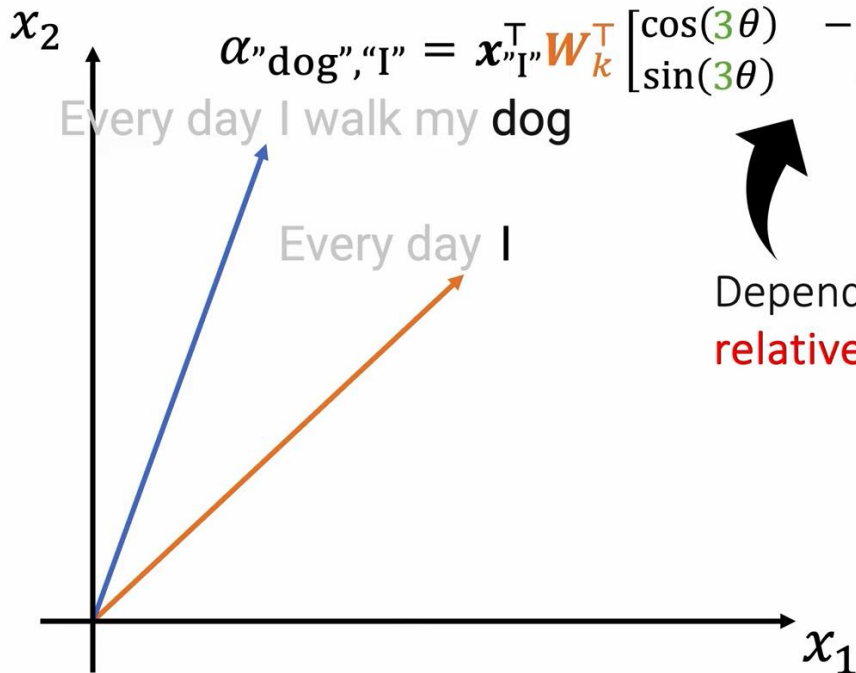
Key vector for "I"

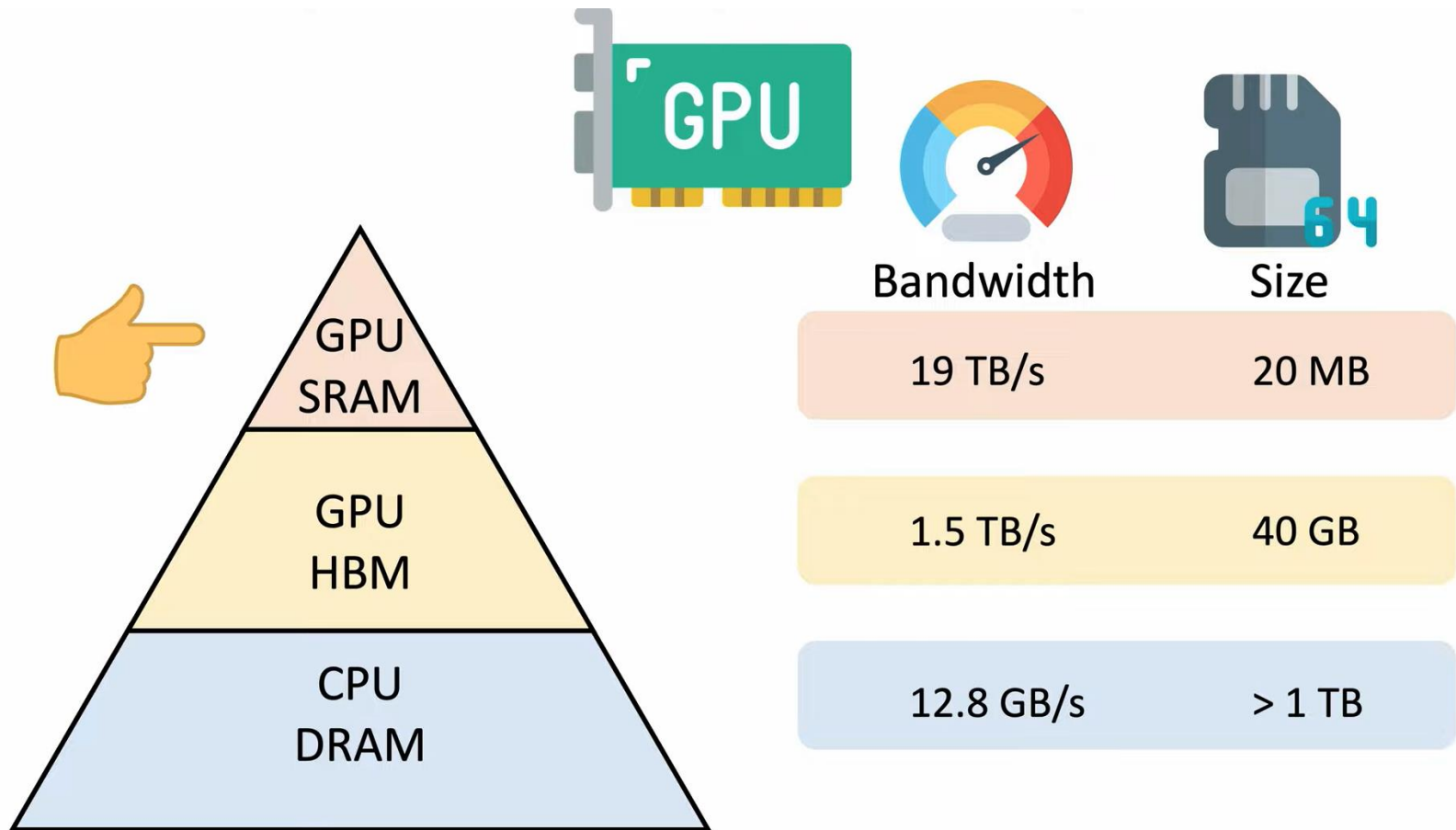
$$\alpha_{\text{"dog","I"}} = x_{\text{"I"}}^T W_k^T \begin{bmatrix} \cos(3\theta) & -\sin(3\theta) \\ \sin(3\theta) & \cos(3\theta) \end{bmatrix} W_q x_{\text{"dog"}}$$

Every day I walk my dog

Every day I

Depends only on the
relative positions

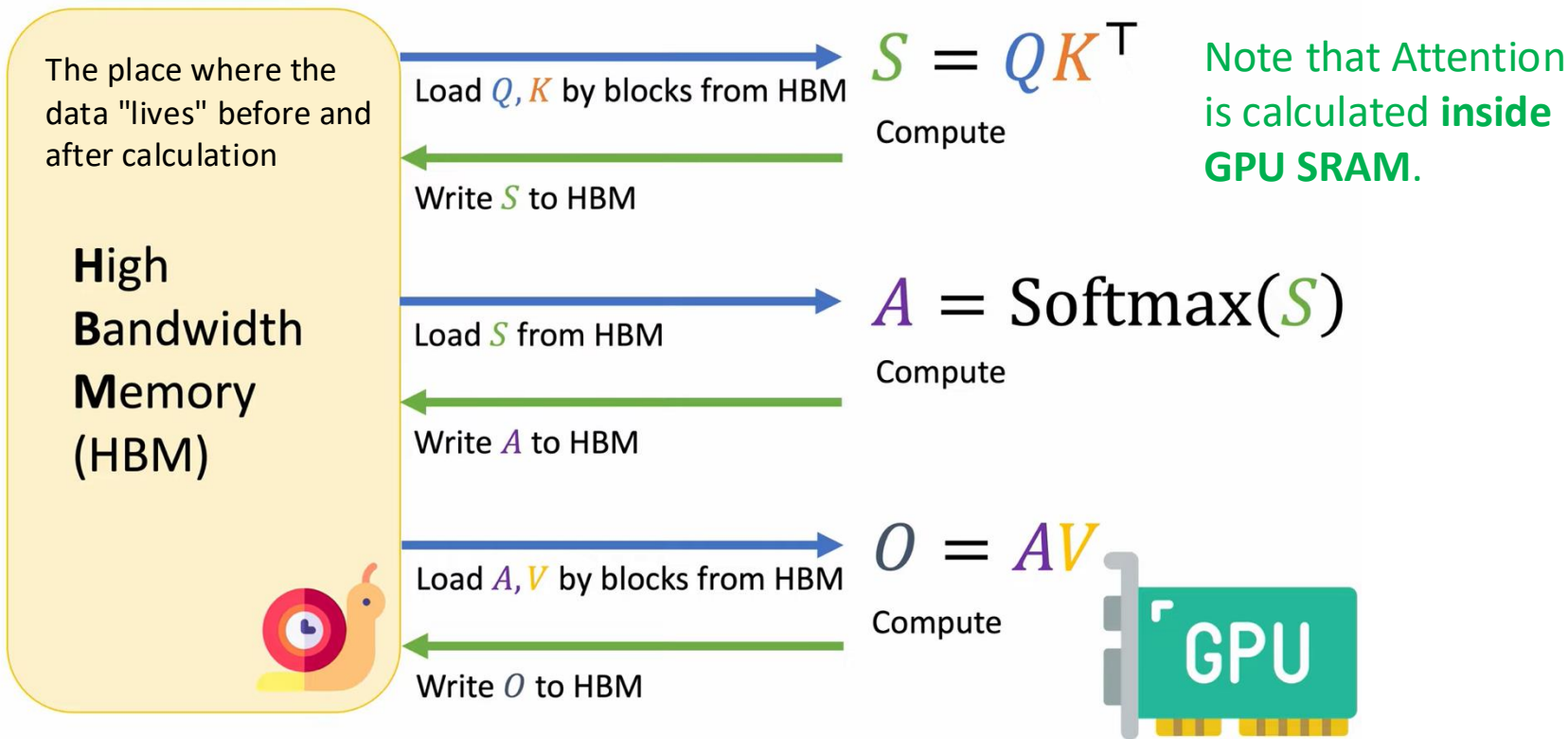




Comparison

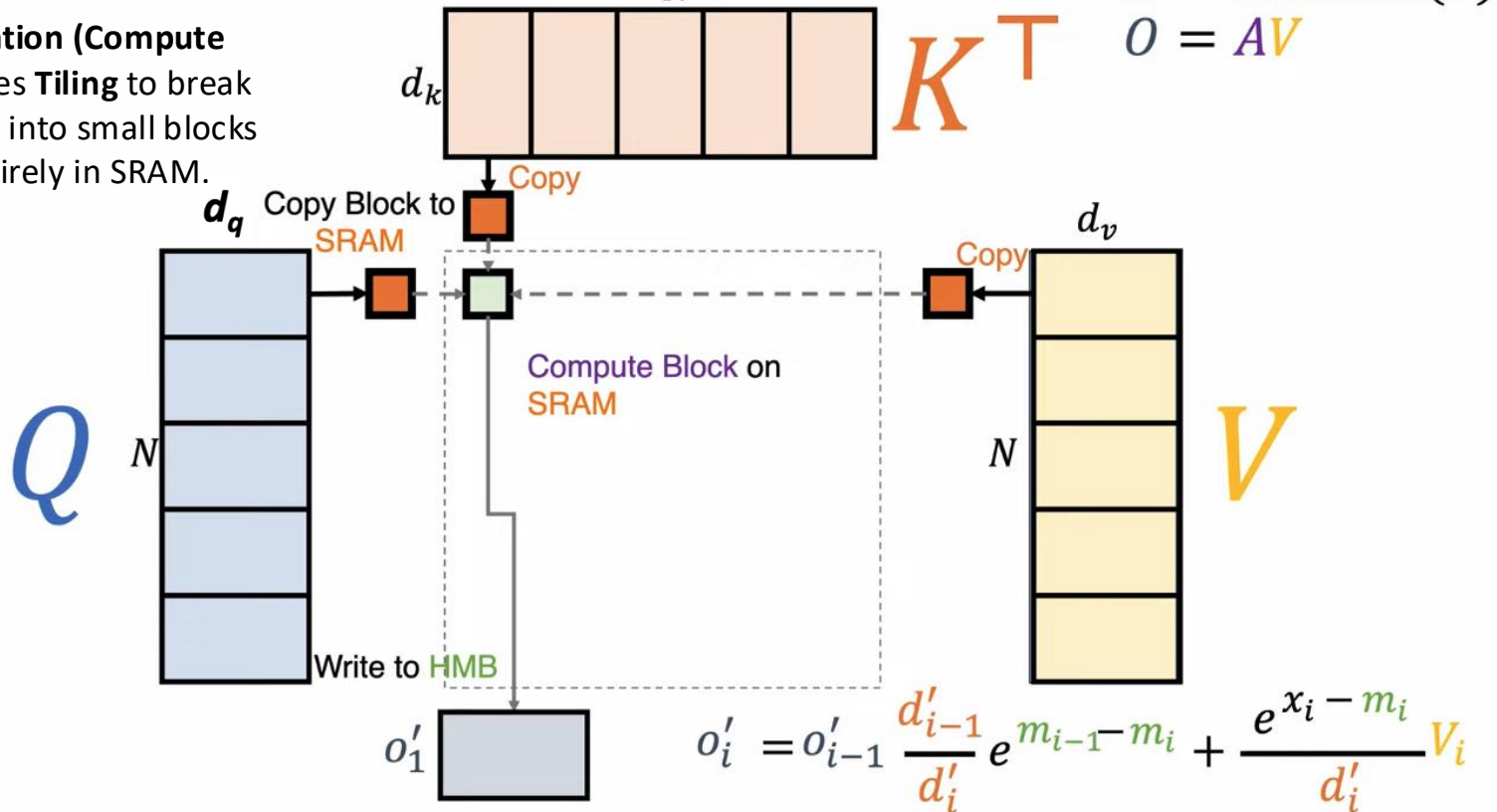
Feature	GPU SRAM	GPU HBM	CPU DRAM
Full Name	Static RAM	High Bandwidth Memory	Dynamic RAM (e.g., DDR5)
Location	On-chip (inside the GPU)	On-package (next to the GPU)	On-motherboard (sockets)
Role	L1/L2 Cache, Registers	Main Video Memory (VRAM)	Main System Memory
Bandwidth	20+ TB/s (Extreme)	1–8 TB/s (Very High)	~50–100 GB/s (Standard)
Capacity	Tiny (KBs to MBs)	Large (16GB to 192GB)	Massive (16GB to 2TB+)
Latency	Lowest (Nanoseconds)	Low	Moderate

Calculating Attention on a GPU



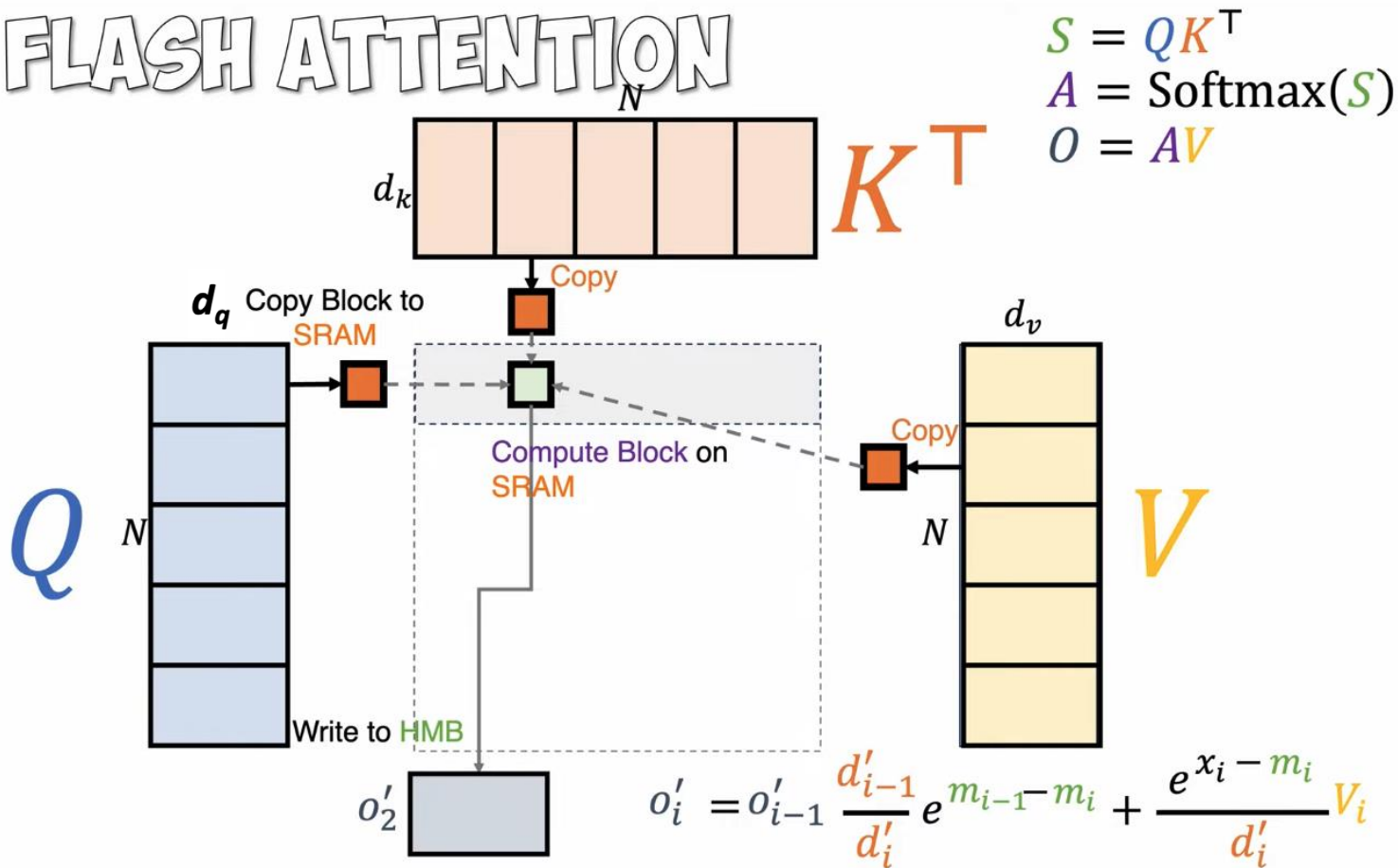
FLASH ATTENTION

FlashAttention (Compute Bound) uses **Tiling** to break the Q, K, V into small blocks that fit entirely in SRAM.





FLASH ATTENTION

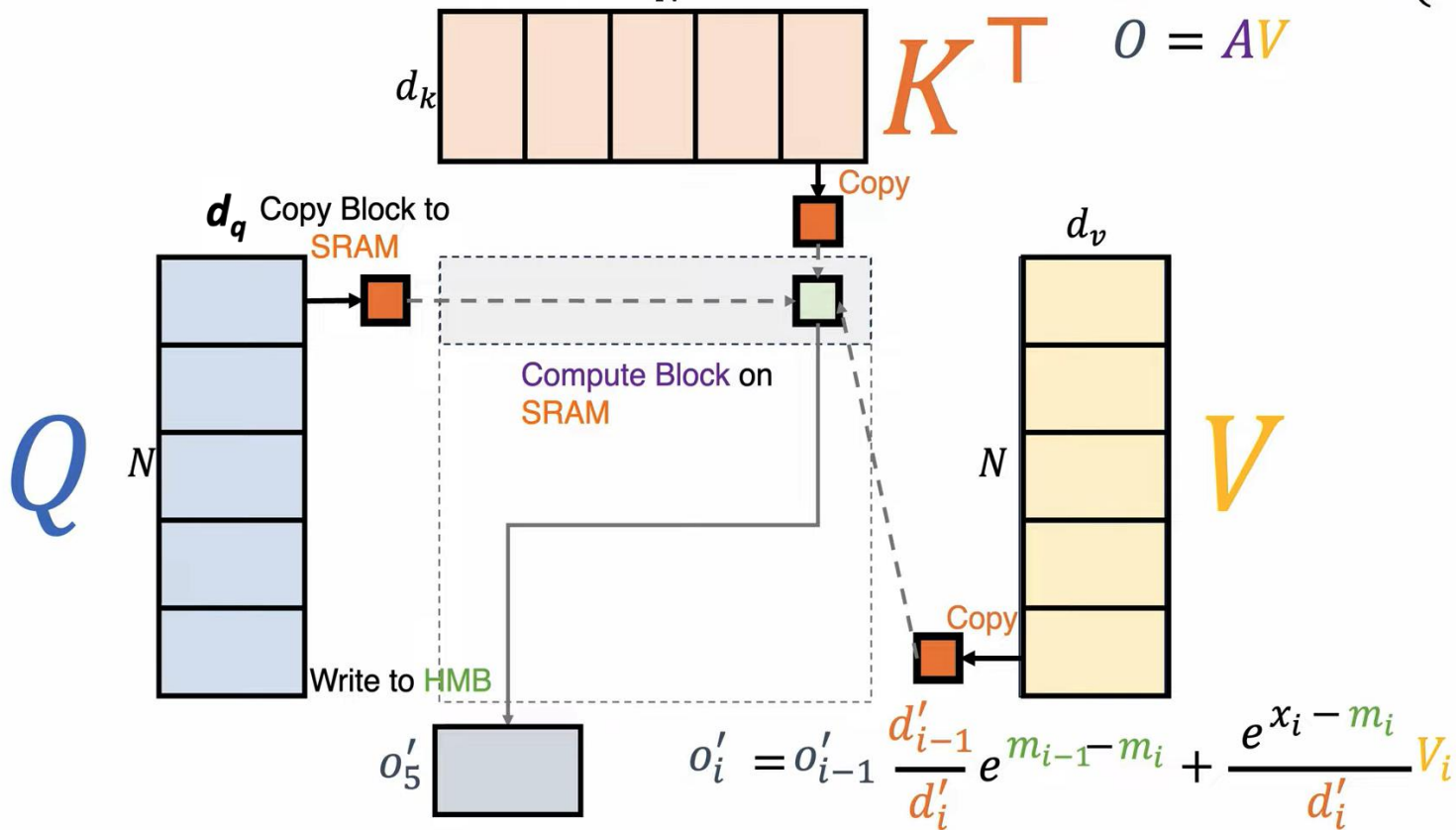


FLASH ATTENTION

$$S = QK^T$$

$$A = \text{Softmax}(S)$$

$$O = AV$$



Linear Attention

- In standard Transformers ("Scaled Dot-Product Attention"), the attention score is calculated by multiplying the Query (Q) and Key (K) matrices first, applying Softmax, and then multiplying by Value (V)

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- Linear attention replaces the Softmax with a kernel feature map, $\phi(\cdot)$, a feature map, often a simple function like ReLU or ELU + 1. This allows us to decompose the similarity score into a simple dot product:

$$(\phi(Q)\phi(K)^T)V = \phi(Q)(\phi(K)^TV)$$

Linear Attention, *Continued*

- Define $S = \phi(K)^T V$ which aggregates the keys and values into a fixed-size $d \times d$ matrix. The matrix S acts as the **memory** (or hidden state) of the model.
- Instead of re-reading the entire history of tokens for every new word (which standard Transformers do), the model just updates S incrementally.
 - Update the States (Write to Memory): $S_t = S_{t-1} + \phi(k_t)^T v_t$
 - Compute the Output (Read from Memory)

$$\text{Output}_t = \frac{\phi(q_t) S_t}{\phi(q_t) Z_t}$$

where the normalization state $Z_t = Z_{t-1} + \phi(k_t)^T$ ensure the values don't explode.

Linear Attention, *Continued*

Feature	Standard Attention (Transformer)	Linear Attention (RWKV, RetNet, Mamba*)
Complexity (Time)	$O(N^2)$ (Quadratic)	$O(N)$ (Linear)
Complexity (Memory)	$O(N^2)$ (Stores attention map)	$O(N)$ (Stores fixed state)
Inference Mode	KV Cache. Must store history of all previous tokens. Cache grows infinitely.	RNN State. Stores a fixed-size state (S). Memory is constant.
Precision	Exact. Softmax is "spiky," allowing perfect recall of specific past tokens.	Approximation. Kernels are "smoother," often causing information "blurring" over time.
Training Parallelism	Yes. Highly parallelizable on GPUs.	Yes. Can compute in parallel (CNN-like) mode during training.
Causal Masking	Easy (Mask the $N \times N$ matrix).	Harder (Requires RNN formulation or chunk wise computation).

Kimi Linear

$(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)$ acts as a correction mechanism that partially **erases or modifies** the existing state $\mathbf{S}_{t-1}$ (memory) to minimize conflict with the new input (essentially preventing the same key from being "over-counted").

$\beta_t \mathbf{k}_t \mathbf{v}_t^\top$ determining how strongly the new key-value pair (k_t, v_t) is written into the memory.

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\alpha_t) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top \in \mathbb{R}^{d_k \times d_v}; \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t \in \mathbb{R}^{d_v}$$

α_t acts as a fine-grained forget gate or decay factor

$$\begin{bmatrix} \text{Matrix} \end{bmatrix} = \left(\begin{bmatrix} \text{Matrix} \end{bmatrix} - \begin{bmatrix} \text{Vector} \end{bmatrix} \times \begin{bmatrix} \text{Vector} \end{bmatrix} \right) \begin{bmatrix} \text{Matrix} \end{bmatrix} + \begin{bmatrix} \text{Vector} \end{bmatrix} \times \begin{bmatrix} \text{Vector} \end{bmatrix} \quad \Bigg| \quad \begin{bmatrix} \text{Vector} \end{bmatrix} = \begin{bmatrix} \text{Matrix} \end{bmatrix} \begin{bmatrix} \text{Vector} \end{bmatrix}$$

Part 3:

Inference Engines

Prefill-Decode (PD) Disaggregation

- PD disaggregation is to separate the computationally distinct prompt processing (prefill) and token generation (decode) phases into dedicated hardware or software instances.
- Disaggregation helps manage competing Service-Level Objectives (SLOs) like Time-to-First-Token (TTFT) and Time-Per-Output-Token (TPOT).
 - The **prefill stage** is generally compute-bound (favors high FLOPs and batch GEMM throughput). It processes the entire input in parallel to build the initial Key-Value (KV) cache.
 - The **decode stage** is typically memory-bandwidth-bound (bottlenecked by KV cache reads) as it generates one token at a time.

Key Idea

- **Prefill Stage (Compute-Bound):** This phase processes the entire input prompt in parallel and builds the Key-Value (KV) cache. It demands high computational power (FLOPs).
- **Decode Stage (Memory-Bound):** This phase autoregressively generates one token at a time, primarily bottlenecked by memory bandwidth as it repeatedly accesses the KV cache.
- **Independent Scaling:** By disaggregating, systems can allocate more GPUs or compute resources to whichever phase is the current bottleneck in a given workload.

Example

- The PD Disaggregation design in SGLang interleaves execution between a Prefill Server and a Decode Server:
 - A Prefill Server and a Decode Server pair via a handshake, establishing a local sender and receiver, respectively.
 - The Decode Server pre-allocates the KV cache, signaling the Prefill Server to begin the model forward pass and compute the KV caches.
 - Once computed, the data transfers to the Decode Server, which handles iterative token generation.

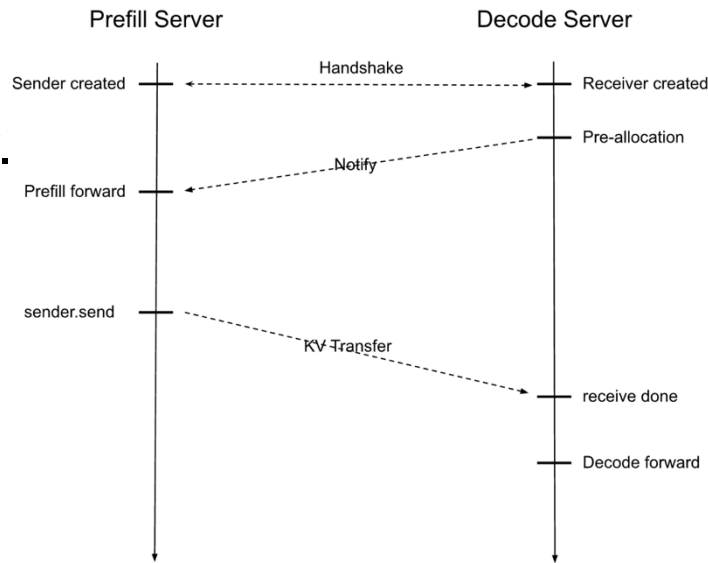
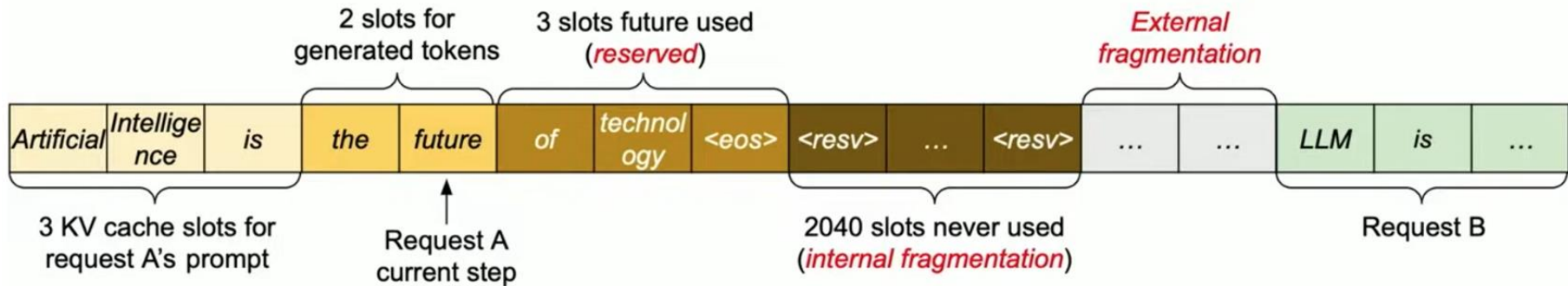


Figure. PD Disaggregation Protocol

Memory Waste in KV Cache

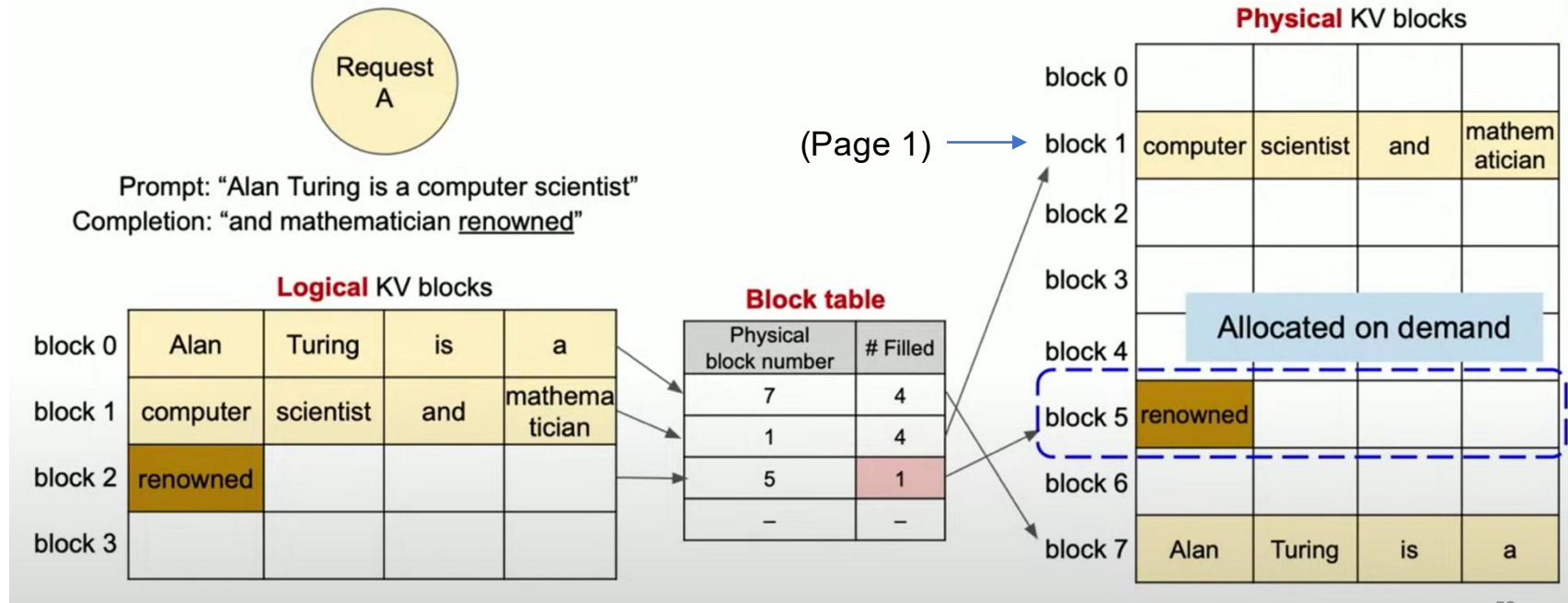


- **Internal fragmentation:** over-allocated due to the unknown output length.
- **Reservation:** not used at the current step, but used in the future
- **External fragmentation:** due to different sequence lengths.

Only **20–40%** of KV cache is utilized to store token states

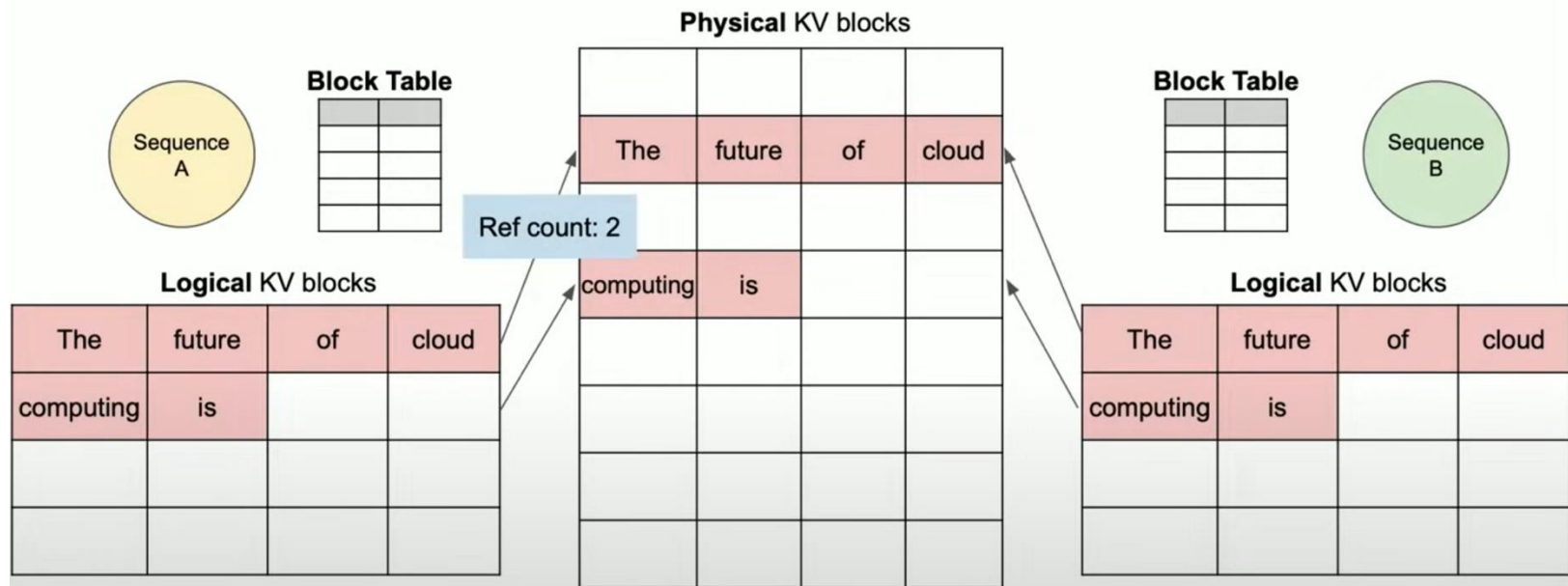
vLLM and PagedAttention

Logical & physical KV blocks



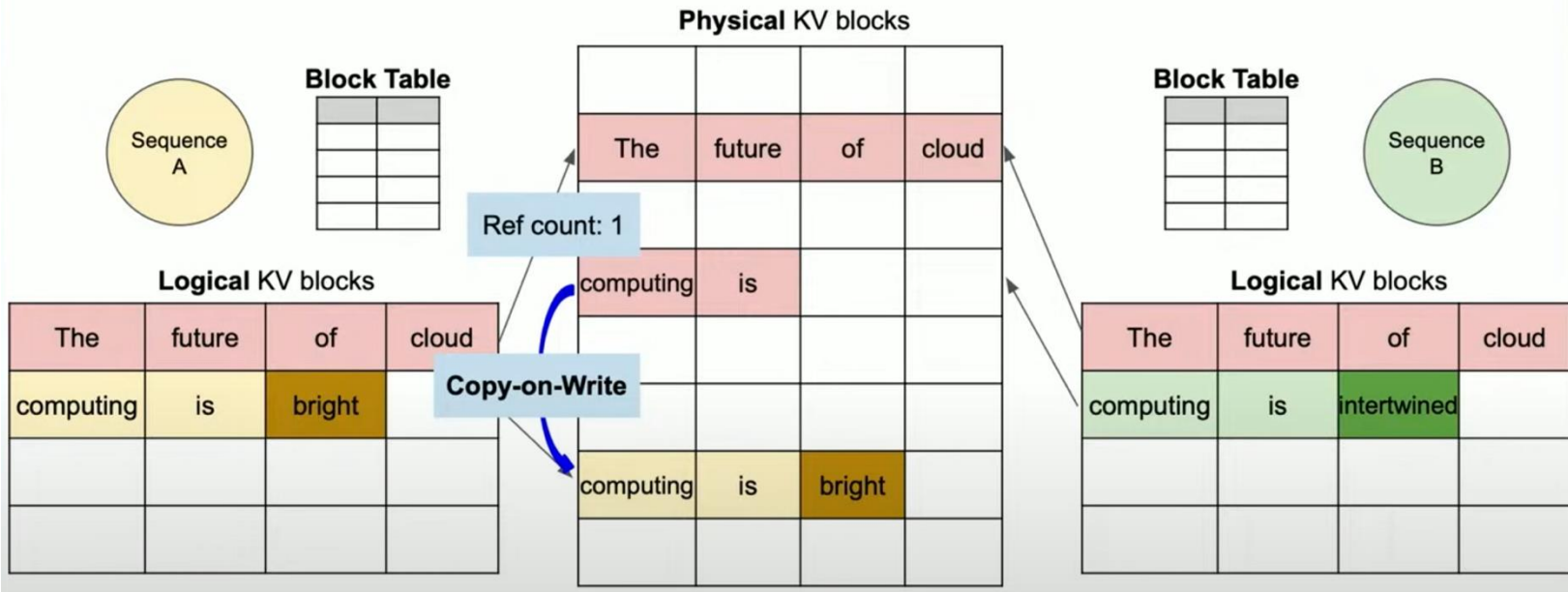
PagedAttention, *Continued*

Sharing KV blocks



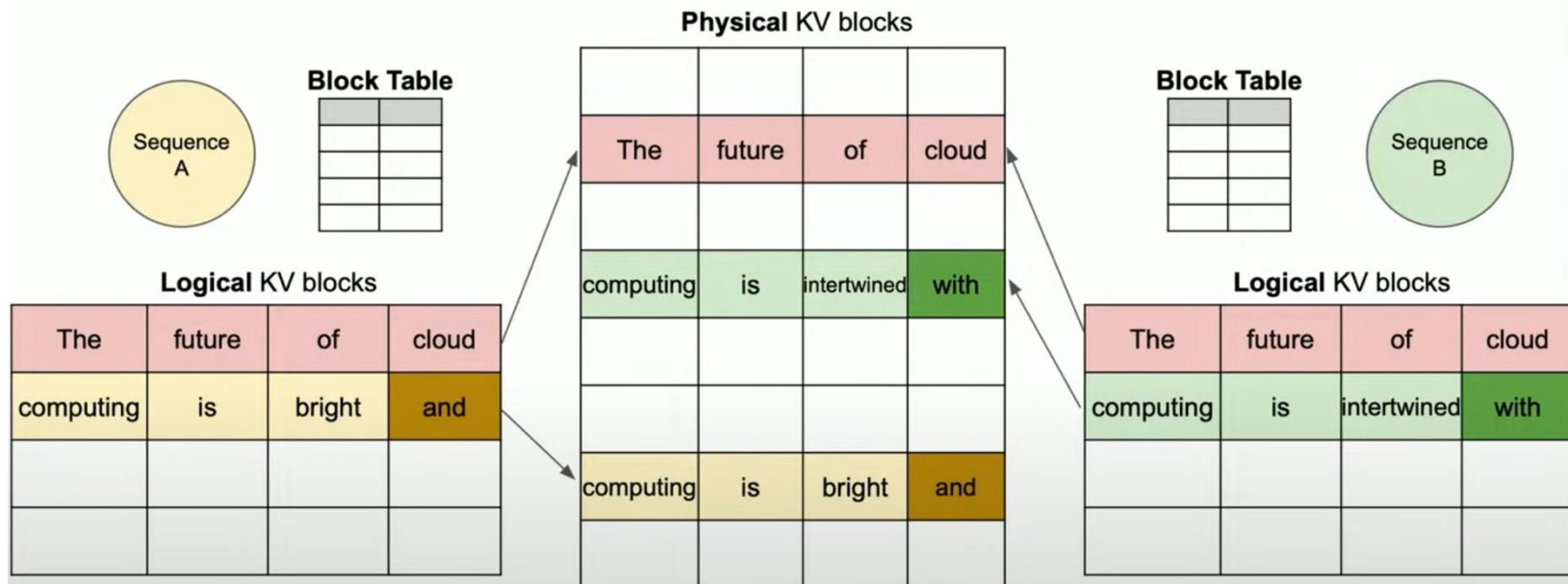
PagedAttention, *Continued*

Sharing KV blocks



PagedAttention, *Continued*

Sharing KV blocks



What is a 'Page' in vLLM?

- In vLLM, a **Page** (often called a **Block**) is a fixed-size chunk of GPU memory reserved to store the Key and Value vectors for a specific number of tokens.
- vLLM treats the massive Key-Value (KV) Cache exactly like an Operating System treats RAM.
 - vLLM breaks a sequence of text (e.g., a user prompt) into fixed-size "pages" so it doesn't need a contiguous block of GPU VRAM.

How do PagedAttention and vLLM benefit LLM Serving?

Reduce memory
fragmentation with
paging

Further reduce
memory usage with
sharing

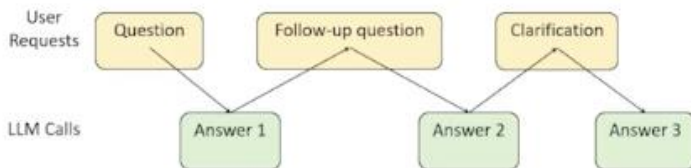
SGLang and RadixAttention

- SGLang uses **PagedAttention** as its underlying memory storage mechanism, but it wraps it inside a more advanced data structure called **RadixAttention**.
 - **Paged Attention** is the "hard drive formatting" (how data is stored)
 - **RadixAttention** is the "file system" (how data is named, organized, and retrieved).

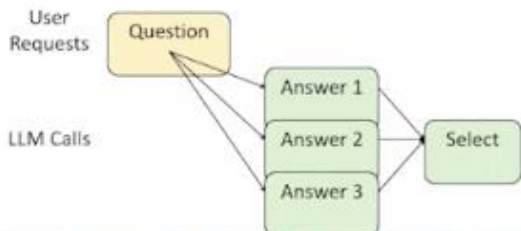
RadixAttention

LLM inference pattern:
Complex pipeline with multiple LLM calls

Chained calls

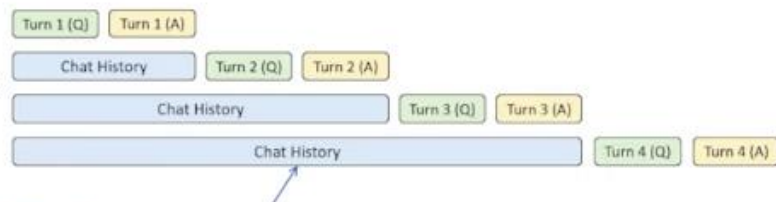


Parallel calls



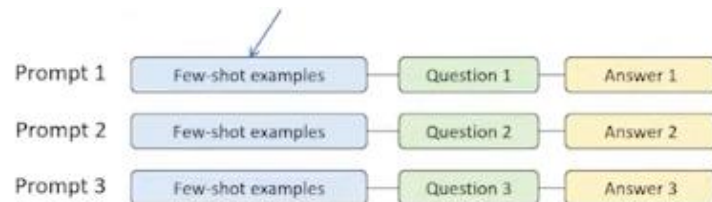
There are rich structures in LLM calls

(a) Multi-turn chat

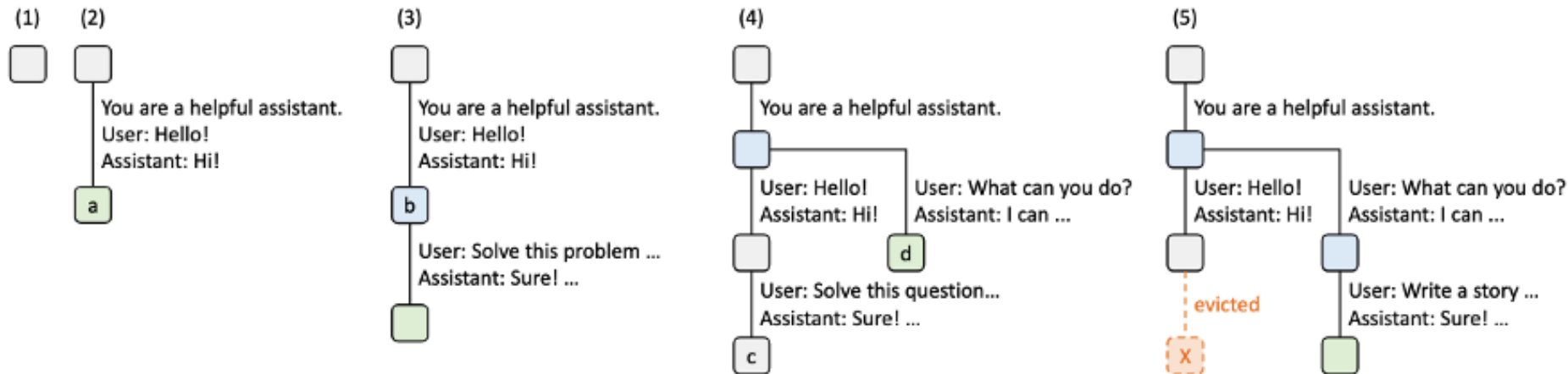


Reusable KV cache (Key-Value cache, some intermediate tensors)

(b) Few-shot learning

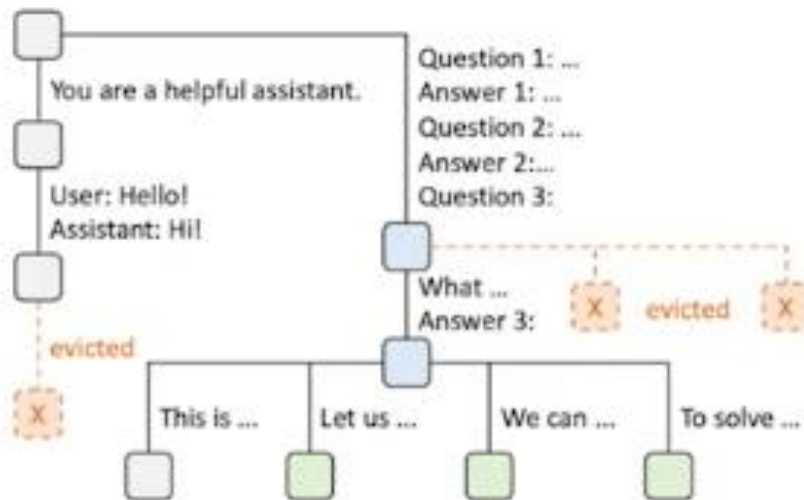


RadixAttention, *Continued*



RadixAttention, *Continued*

RadixAttention handles complex reuse patterns



RadixAttention enables
efficient prefix matching,
insertion, and eviction.

It handles trees with hundreds of
thousands of tokens.

Cache-aware Scheduling Increase Cache Hit Rate

- Idea: utilize user annotation & runtime metrics for scheduling
- You have a line of three people waiting:
 1. Alice: Ask a question about *Harry Potter*.
 2. Bob: Ask a question about *The Hobbit*.
 3. Charlie: Ask a question about *Harry Potter*.
- **W/O Cache-Aware Scheduling (First-Come-First-Served)**
 - Result: You had to fetch and open books **3 times**.
- **With SGLang Cache-Aware Scheduling**
 - **Result:** You only had to fetch and open books **2 times**.

vLLM vs SGLang

- While standard vLLM uses PagedAttention primarily to fit more concurrent users into memory (throughput), SGLang uses the stability of these pages to create a persistent, searchable **KV Cache memory bank**.
- SGLang leverages PagedAttention by changing the **lifecycle** and **management** of the pages.
 - SGLang builds a data structure called **RadixAttention** on top of the PagedAttention blocks.
 - When a request finishes, SGLang **does not free** the blocks. Instead, it marks them as "evictable" but keeps them in a Least Recently Used (LRU) cache.

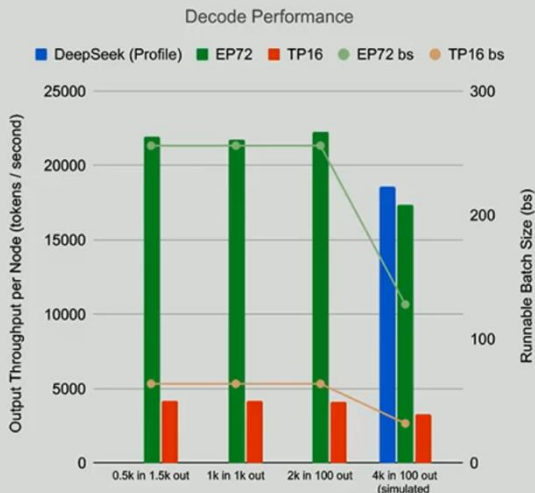
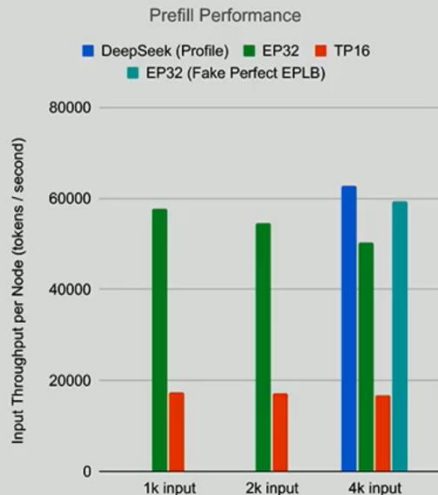
SGLang Performance

Benchmark results

Large-scale EP + PD disaggregation achieves
3x higher prefill throughput and 5x higher decode throughput

EP32: Expert Parallelism (EP) with 32 Experts

TP16: 16-way tensor parallelism



Throughputs of prefill (P) and decode (D) phases are evaluated independently, assuming unlimited resources for the non-tested phase to isolate and maximize the load on the tested nodes—mirroring the setup used by DeepSeek.

Key Intuitions

- **Load Balancing is Critical:** The "fake perfect" baseline shows that DeepSeek's main advantage comes from their **Expert Parallelism Load Balancing (EPLB)**, which prevents specific GPUs from becoming bottlenecks when certain experts are used more than others
- **EP > TP for MoE:** For a model with as many experts as DeepSeek-V3, Expert Parallelism (EP) is significantly more efficient than Tensor Parallelism (TP) because it minimizes the amount of data that must be synchronized across GPUs during the MoE layer computation

Comparison

Feature	vLLM	SGLang
Primary Mechanism	PagedAttention	RadixAttention
Best Use Case	Public Chatbots, Batch Processing	AI Agents, Tool Use, Coding Assistants
Throughput (Simple)	Higher (for simple, short prompts)	Competitive (slightly lower overhead)
Throughput (Complex)	Lower (re-computes prefixes often)	Higher (reuses prefix cache aggressively)
Structured Output	Basic (Guidance/Outlines integration)	Native & Fast (Compressed Finite State Machine)
Multi-Turn Cache	Prefix Caching (Manual/Hash-based)	Automatic (Tree-based caching)
Developer Exp.	"Set and Forget" (Standard API)	Requires learning SGLang DSL for max benefit

TensorRT

- **NVIDIA TensorRT (The Chef)** is a software development kit (SDK) for high-performance deep learning inference. It essentially acts as a **compiler** for your neural networks, taking a trained model and optimizing it to run as fast as possible on NVIDIA GPUs.
- **Key Optimizations (The "Magic"):**
 - **Layer & Tensor Fusion:** fuses nodes in the kernel graph to reduce overhead
 - **Precision Calibration (Quantization):** reduce precision from FP32 (32-bit floating point) to FP16 or INT8 (8-bit integer).
 - **Kernel Auto-Tuning:** picks the absolute fastest implementation for that specific target GPU
 - **Dynamic Tensor Memory:** minimizes the memory footprint by designating memory for tensors only for the duration they are needed,
 - **Multi-Stream Execution:** processes multiple input streams in parallel

TensorRT-LLM and Dynamo

- **TensorRT-LLM** (The Kitchen Manager) is a specialized library that includes:
 - **FlashAttention integration**: Optimized attention kernels.
 - **PagedAttention**: Efficient memory management for the KV cache
 - **Tensor Parallelism**: Splitting the model across multiple GPUs automatically.
- **NVIDIA Dynamo** (The Orchestrator) is a serving framework designed for Data Center Scale inference, specifically for Generative AI and LLMs.

Comparison

Feature	TensorRT	TensorRT-LLM	NVIDIA Dynamo
Primary Function	Compiler. Optimizes neural network graphs for hardware.	Library. Implements LLM-specific features (Attention, KV Cache) using TensorRT.	Orchestrator. Manages traffic, routing, and resources across a fleet of GPUs.
Key "Magic"	Kernel Fusion. Merging math operations to reduce memory overhead.	In-Flight Batching. Processing different requests at different speeds simultaneously.	Disaggregated Serving. Splitting "Prefill" (Input processing) and "Decode" (Generation) onto different GPUs.
Input	ONNX Model / PyTorch Graph	Large Language Model Weights (Llama 3, GPT)	HTTP/gRPC Requests (User Traffic)
Scale	1 GPU	1 to 8 GPUs (Tensor Parallelism)	1,000+ GPUs (Data Center Scale)
You use it when...	You are building a custom neural net and need raw speed.	You are deploying an LLM and need standard optimizations (Attention, Batching).	You are running a service like ChatGPT or Perplexity and need to balance load across thousands of users.